

Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 26: Deep Learning

Recurrent Neural Networks (RNNs) include feedback from one layer to another and they can typically be trained by unfolding (or unrolling) the recurrent connections, resulting in deep networks whose parameters can be learned via the backpropagation algorithm.

Since RNNs are prone to the vanishing and exploding gradients problem, we next consider *gated RNNs* that introduce a new type of layer that endows the network with the ability to selectively read, store, and write the hidden state via an internal *memory layer*. Gated RNNs are highly effective for prediction tasks on sequence inputs.

Convolutional Neural Networks (CNNs) that are deep MLPs that exploit spatial or temporal relationships between different elements of each layer to construct a hierarchy of *features* that can be used for regression or classification tasks. Unlike regular MLPs whose layers are fully connected, CNNs have layers that are localized and sparse. In particular, CNNs are highly effective for image inputs.

Recurrent Neural Networks

Multilayer perceptrons are feed-forward networks in which the information flows in only one direction, namely from the input layer to the output layer via the hidden layers.

In contrast, recurrent neural networks (RNNs) are dynamically driven (e.g., temporal), with a *feedback* loop between two (or more) layers, which makes them ideal for learning from sequence data.

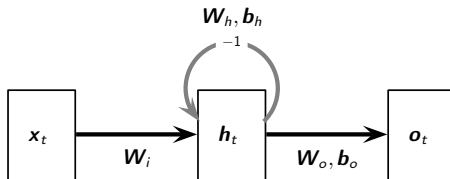
The task of an RNN is to learn a function that predicts the target sequence $\mathcal{Y}$ given the input sequence $\mathcal{X}$. That is, the predicted output $\mathbf{o}_t$ on input $\mathbf{x}_t$ should be similar or close to the target response $\mathbf{y}_t$, for each time point t .

Recurrent Neural Network

To learn dependencies between elements of the input sequence, an RNN maintains a sequence of m -dimensional hidden state vectors $\mathbf{h}_t \in \mathbb{R}^m$, where $\mathbf{h}_t$ captures the essential features of the input sequences up to time t . The hidden vector $\mathbf{h}_t$ at time t depends on the input vector $\mathbf{x}_t$ at time t and the previous hidden state vector $\mathbf{h}_{t-1}$ from time $t-1$, and it is computed as follows:

$$\mathbf{h}_t = f^h(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}_h)$$

Here, f^h is the hidden state activation function, typically tanh or ReLU.



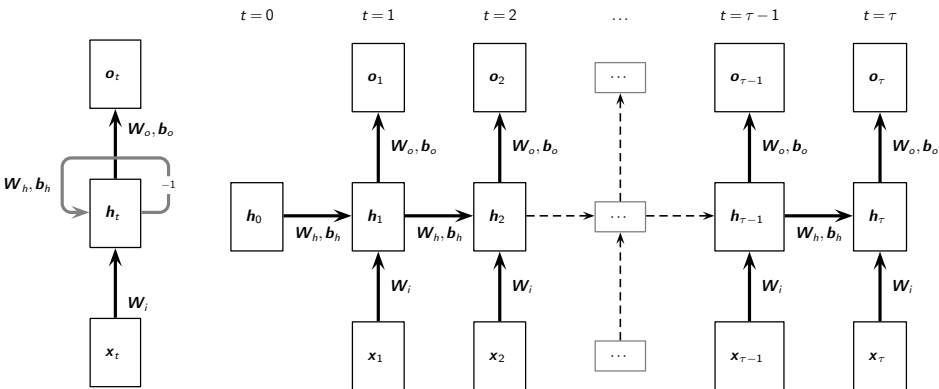
It is important to note that all the weight matrices and bias vectors are *independent* of the time t . For example, for the hidden layer, the same weight matrix $\mathbf{W}_h$ and bias vector $\mathbf{b}_h$ is used and updated while training the model, over all time steps t .

This is an example of *parameter sharing* or *weight tying* between different layers or components of a neural network. Likewise, the input weight matrix $\mathbf{W}_i$, the output weight matrix $\mathbf{W}_o$ and the bias vector $\mathbf{b}_o$ are all shared across time.

This greatly reduces the number of parameters that need to be learned by the RNN, but it also relies on the assumption that all relevant sequential features can be captured by the shared parameters.

RNN unfolded in time.

We can observe that the feed-back loop has been unfolded for τ time steps. We can also explicitly observe the parameter sharing across time, since all weight matrices and bias vectors are independent of t .



Feed-forward in Time

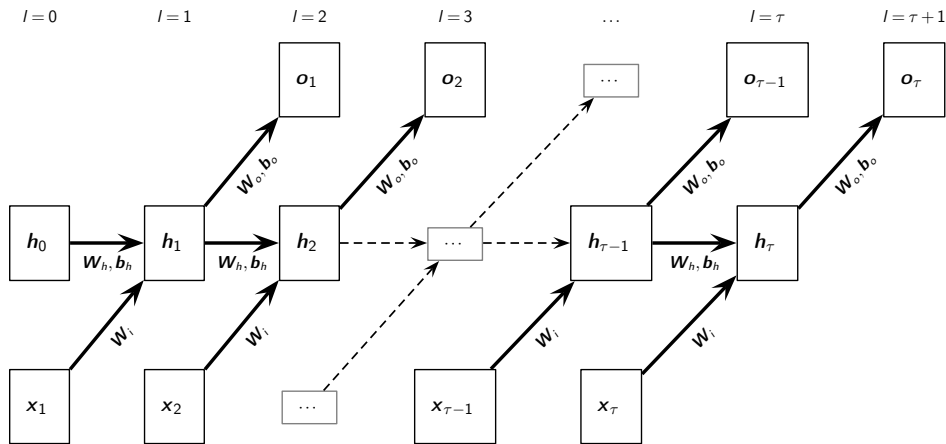
The feed-forward process starts at time $t = 0$, taking as input the initial hidden state vector $\mathbf{h}_0$, which is usually set to 0 or it can be user-specified, say from a previous prediction step.

Given the current set of parameters, we predict the output $\mathbf{o}_t$ at each time step $t = 1, 2, \dots, \tau$.

$$\begin{aligned}\mathbf{o}_t &= f^o(\mathbf{W}_o^T \mathbf{h}_t + \mathbf{b}_o) \\ &= f^o\left(\mathbf{W}_o^T \underbrace{f^h(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}_h)}_{\mathbf{h}_t} + \mathbf{b}_o\right) \\ &= \quad \vdots \\ &= f^o\left(\mathbf{W}_o^T f^h\left(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{W}_h^T f^h\left(\dots f^h\left(\underbrace{\mathbf{W}_i^T \mathbf{x}_1 + \mathbf{W}_h^T \mathbf{h}_0 + \mathbf{b}_h}_{\mathbf{h}_1}\right) + \dots\right) + \mathbf{b}_h\right) + \mathbf{b}_o\right)\end{aligned}$$

We can observe that the RNN implicitly makes a prediction for every prefix of the input sequence, since $\mathbf{o}_t$ depends on all the previous input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$, but not on any future inputs $\mathbf{x}_{t+1}, \dots, \mathbf{x}_\tau$.

RNN: Feed-forward step



Training an RNN

For training the network, we compute the error or *loss* between the predicted and response vectors over all time steps. For example, the squared error loss is given as

$$\mathcal{E}_{\mathcal{X}} = \sum_{t=1}^{\tau} \mathcal{E}_{\mathbf{x}_t} = \frac{1}{2} \cdot \sum_{t=1}^{\tau} \|\mathbf{y}_t - \mathbf{o}_t\|^2$$

On the other hand, if we use a softmax activation at the output layer, then we use the cross-entropy loss, given as

$$\mathcal{E}_{\mathcal{X}} = \sum_{t=1}^{\tau} \mathcal{E}_{\mathbf{x}_t} = - \sum_{t=1}^{\tau} \sum_{i=1}^p y_{ti} \cdot \ln(o_{ti})$$

where $\mathbf{y}_t = (y_{t1}, y_{t2}, \dots, y_{tp})^T \in \mathbb{R}^p$ and $\mathbf{o}_t = (o_{t1}, o_{t2}, \dots, o_{tp})^T \in \mathbb{R}^p$.

Backpropagation in Time

Once the the output sequence $\mathcal{O} = \langle \mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_T \rangle$ is generated, we can compute the error in the predictions using the squared error (or cross-entropy) loss function, which can in turn be used to compute the net gradient vectors that are backpropagated from the output layers to the input layers for each time step.

Let $\mathcal{E}_{\mathbf{x}_t}$ denote the loss on $\mathbf{x}_t$ from the input sequence $\mathcal{X} = \langle \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T \rangle$.

Define δ_t^o as the net gradient vector for the output vector $\mathbf{o}_t$, i.e., the derivative of the error function $\mathcal{E}_{\mathbf{x}_t}$ with respect to the net value at each neuron in $\mathbf{o}_t$, given as

$$\delta_t^o = \left(\frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{t1}^o}, \frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{t2}^o}, \dots, \frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{tp}^o} \right)^T$$

where $\mathbf{o}_t = (o_{t1}, o_{t2}, \dots, o_{tp})^T \in \mathbb{R}^p$ is the p -dimensional output vector at time t , and net_{ti}^o is the net value at output neuron o_{ti} at time t .

Backpropagation in Time

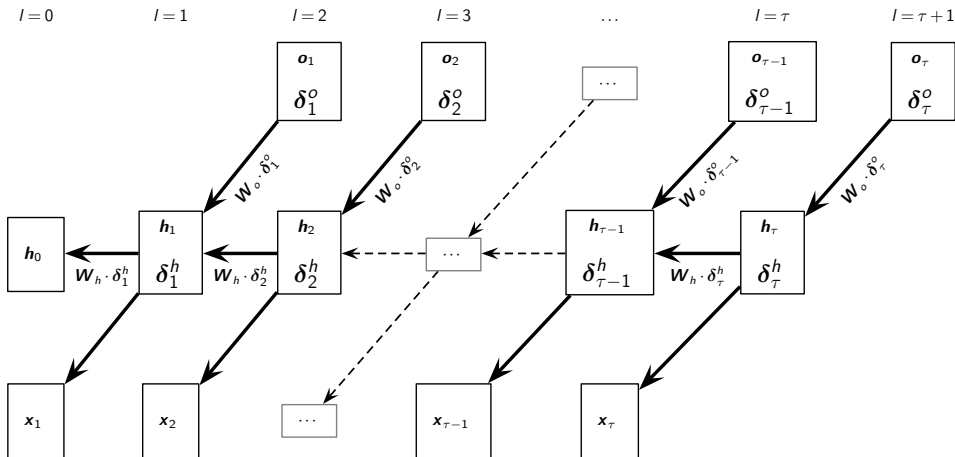
Likewise, let δ_t^h denote the net gradient vector for the hidden state neurons $\mathbf{h}_t$ at time t

$$\delta_t^h = \left(\frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{t1}^h}, \frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{t2}^h}, \dots, \frac{\partial \mathcal{E}_{\mathbf{x}_t}}{\partial \text{net}_{tm}^h} \right)^T$$

where $\mathbf{h}_t = (h_{t1}, h_{t2}, \dots, h_{tm})^T \in \mathbb{R}^m$ is the m -dimensional hidden state vector at time t , and net_{ti}^h is the net value at hidden neuron h_{ti} at time t .

The key step in backpropagation is to compute the net gradients in reverse order, starting from the output neurons to the input neurons via the hidden neurons.

RNN: Backpropagation step



Computing Net Gradients

The backpropagation step reverses the flow direction for computing the net gradients δ_t^o and δ_t^h . The net gradient vector at the output $\mathbf{o}_t$ can be computed as follows:

$$\delta_t^o = \partial \mathbf{f}_t^o \odot \partial \mathcal{E}_{x_t}$$

where $\odot$ is the element-wise or Hadamard product.

On the other hand, the net gradients at each of the hidden layers need to account for the incoming net gradients from $\mathbf{o}_t$ and from $\mathbf{h}_{t+1}$.

$$\delta_t^h = \partial \mathbf{f}_t^h \odot \left((\mathbf{w}_o \cdot \delta_t^o) + (\mathbf{w}_h \cdot \delta_{t+1}^h) \right)$$

Note that for $\mathbf{h}_\tau$, it depends only on $\mathbf{o}_\tau$. The net gradients do not have to be computed for $\mathbf{h}_0$ or for any of the input neurons $\mathbf{x}_t$, since these are leaf nodes in the backpropagation graph.

We use an RNN to learn the Reber grammar, which is generated according to the automata. Let $\Sigma = \{B, E, P, S, T, V, X\}$ denote the alphabet comprising the seven symbols. Further, let $\$$ denote a terminal symbol.

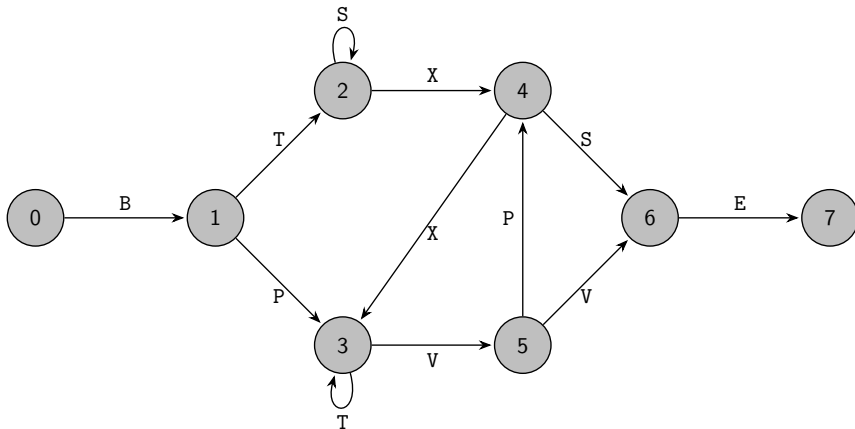
Starting from the initial node, we can generate strings that follow the Reber grammar by emitting the symbols on the edges. If there are two transitions out of a node, each one can be chosen with equal probability.

The task of the RNN is to learn to predict the next symbol for each of the positions in a given Reber sequence. For training, we generate Reber sequences from the automata.

RNN

Reber grammar automata.

The sequence $\langle B, T, S, S, X, X, T, V, V, E \rangle$ is a valid Reber sequence (with the corresponding state sequence $\langle 0, 1, 2, 2, 2, 4, 3, 3, 5, 6, 7 \rangle$). On the other hand, the sequence $\langle B, P, T, X, S, E \rangle$ is not a valid Reber sequence, since there is no edge out of state 3 with the symbol X.



Let $S_{\mathcal{X}} = \langle s_1, s_2, \dots, s_T \rangle$ be a Reber sequence. The corresponding true output $\mathcal{Y}$ is then given as the set of next symbols from each of the edges leaving the state corresponding to each position in $S_{\mathcal{X}}$.

For example, consider the Reber sequence $S_{\mathcal{X}} = \langle B, P, T, V, V, E \rangle$, with the state sequence $\pi = \langle 0, 1, 3, 3, 5, 6, 7 \rangle$. The desired output sequence is then given as $S_{\mathcal{Y}} = \{P|T, T|V, T|V, P|V, E, \$\}$, where $\$$ is the terminal symbol.

Here, $P|T$ denotes that the next symbol can be either P or T . We can see that $S_{\mathcal{Y}}$ comprises the sequence of possible next symbols from each of the states in π (excluding the start state 0).

To generate the training data for the RNN, we have to convert the symbolic Reber strings into numeric vectors through binary encoding:

B	$(1, 0, 0, 0, 0, 0, 0)^T$
E	$(0, 1, 0, 0, 0, 0, 0)^T$
P	$(0, 0, 1, 0, 0, 0, 0)^T$
S	$(0, 0, 0, 1, 0, 0, 0)^T$
T	$(0, 0, 0, 0, 1, 0, 0)^T$
V	$(0, 0, 0, 0, 0, 1, 0)^T$
X	$(0, 0, 0, 0, 0, 0, 1)^T$
\$	$(0, 0, 0, 0, 0, 0, 0)^T$

The terminal symbol \$ is not part of the alphabet, and its encoding is all 0's.

Finally, to encode the possible next symbols, we follow a similar binary encoding with a 1 in the column corresponding to the allowed symbols.

For example, the choice P|T is encoded as $(0,0,1,0,1,0,0)^T$. Thus, the Reber sequence $S_{\mathcal{X}}$ and the desired output sequence $S_{\mathcal{Y}}$ are encoded as:

	$\mathcal{X}$						$\mathcal{Y}$					
	x_1	x_2	x_3	x_4	x_5	x_6	y_1	y_2	y_3	y_4	y_5	y_6
Σ	B	P	T	V	V	E	P T	T V	T V	P V	E	\$
B	1	0	0	0	0	0	0	0	0	0	0	0
E	0	0	0	0	0	1	0	0	0	0	1	0
P	0	1	0	0	0	0	1	0	0	1	0	0
S	0	0	0	0	0	0	0	0	0	0	0	0
T	0	0	1	0	0	0	1	1	1	0	0	0
V	0	0	0	1	1	0	0	1	1	1	0	0
X	0	0	0	0	0	0	0	0	0	0	0	0

For training, we generate $n = 400$ Reber sequences with a minimum length of 30. The maximum sequence length is $\tau = 52$. Each of these Reber sequences is used to create a training pair $(\mathcal{X}, \mathcal{Y})$ as described above.

Next, we train an RNN with $m = 4$ hidden neurons using tanh activation. The input and output layer sizes are determined by the dimensionality of the encoding, namely $d = 7$ and $p = 7$. We use a sigmoid activation at the output layer, treating each neuron as independent. We use the binary cross entropy error function.

The RNN is trained for $r = 10000$ epochs, using gradient step size $\eta = 1$ and the entire set of 400 input sequences as the batch size. The RNN model learns the training data perfectly, making no errors in the prediction of the set of possible next symbols.

We test the RNN model on 100 previously unseen Reber sequences (with minimum length 30, as before). The RNN makes no errors on the test sequences.

On the other hand, we also trained an MLP with a single hidden layer, with size m varying between 4 and 100.

Even after $r = 10000$ epochs, the MLP is not able to correctly predict any of the output sequences perfectly.

It makes 2.62 mistakes on average per sequence for both the training and testing data.

Increasing the number of epochs or the number of hidden layers does not improve the MLP performance.

Bidirectional RNNs

An RNN makes use of a hidden state $\mathbf{h}_t$ that depends on the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$ at time t . In other words, it only looks at information from the past.

A bidirectional RNN (BRNN) extends the RNN model to also include information from the future.

In particular, a BRNN maintains a backward hidden state vector $\mathbf{b}_t \in \mathbb{R}^m$ that depends on the next backward hidden state $\mathbf{b}_{t+1}$ and the current input $\mathbf{x}_t$. The output at time t is a function of both $\mathbf{h}_t$ and $\mathbf{b}_t$. In particular, we compute the forward and backward hidden state vectors as follows:

$$\mathbf{h}_t = f^h(\mathbf{W}_{ih}^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{b}_t = f^b(\mathbf{W}_{ib}^T \mathbf{x}_t + \mathbf{W}_b^T \mathbf{b}_{t+1} + \mathbf{b}_b)$$

The output at time t is computed only when both $\mathbf{h}_t$ and $\mathbf{b}_t$ are available, and is given as

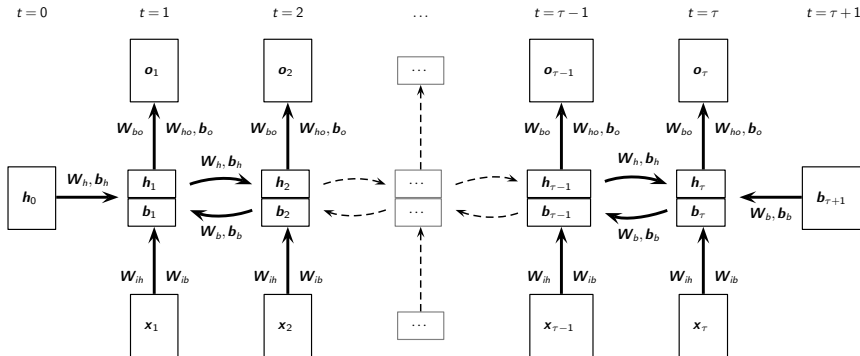
$$\mathbf{o}_t = f^o(\mathbf{W}_{ho}^T \mathbf{h}_t + \mathbf{W}_{bo}^T \mathbf{b}_t + \mathbf{b}_o)$$

It is clear that BRNNs need the complete input before they can compute the output.

We can also view a BRNN as having two sets of input sequences, namely the forward input sequence $\mathcal{X} = \langle \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_\tau \rangle$ and the reversed input sequence $\mathcal{X}^r = \langle \mathbf{x}_\tau, \mathbf{x}_{\tau-1}, \dots, \mathbf{x}_1 \rangle$, with the corresponding hidden states $\mathbf{h}_t$ and $\mathbf{b}_t$, which together determine the output $\mathbf{o}_t$.

Thus, a BRNN is comprised of two “stacked” RNNs with independent hidden layers that jointly determine the output.

Bidirectional RNN: Unfolded in time.



Gated RNNs: Long Short-Term Memory Networks

One of the problems in training RNNs is their susceptibility to either the *vanishing gradient* or the *exploding gradient* problem. For example, consider the task of computing the net gradient vector δ_t^h for the hidden layer at time t , given as

$$\delta_t^h = \partial \mathbf{f}_t^h \odot \left((\mathbf{W}_o \cdot \delta_t^o) + (\mathbf{W}_h \cdot \delta_{t+1}^h) \right)$$

Assume for simplicity that we use a linear activation function, i.e., $\partial \mathbf{f}_t^h = 1$, and let us ignore the net gradient vector for the output layer, focusing only on the dependence on the hidden layers. Then for an input sequence of length τ , we have

$$\delta_t^h = \mathbf{W}_h \cdot \delta_{t+1}^h = \mathbf{W}_h (\mathbf{W}_h \cdot \delta_{t+2}^h) = \mathbf{W}_h^2 \cdot \delta_{t+2}^h = \dots = \mathbf{W}_h^{\tau-t} \cdot \delta_\tau^h$$

We can observe that the net gradient from time τ affects the net gradient vector at time t as a function of $\mathbf{W}_h^{\tau-t}$, i.e., as powers of the hidden weight matrix $\mathbf{W}_h$.

Gated RNNs: Long Short-Term Memory Networks

Let the *spectral radius* of $\mathbf{W}_h$, defined as the absolute value of its largest eigenvalue, be given as $|\lambda_1|$.

It turns out that if $|\lambda_1| < 1$, then $\|\mathbf{W}_h^k\| \rightarrow 0$ as $k \rightarrow \infty$, that is, the gradients vanish as we train on long sequences.

On the other hand, if $|\lambda_1| > 1$, then at least one element of $\mathbf{W}_h^k$ becomes unbounded and thus $\|\mathbf{W}_h^k\| \rightarrow \infty$ as $k \rightarrow \infty$, that is, the gradients explode as we train on long sequences.

It is clear that the net gradients scale according to the eigenvalues of $\mathbf{W}_h$. Therefore, if $|\lambda_1| < 1$, then $|\lambda_1|^k \rightarrow 0$ as $k \rightarrow \infty$, and since $|\lambda_1| \geq |\lambda_i|$ for all $i = 1, 2, \dots, m$, then necessarily $|\lambda_i|^k \rightarrow 0$ as well. That is, the gradients vanish.

On the other hand, if $|\lambda_1| > 1$, then $|\lambda_1|^k \rightarrow \infty$ as $k \rightarrow \infty$, and the gradients explode.

Therefore, for the error to neither vanish nor explode, the spectral radius of $\mathbf{W}_h$ should remain 1 or very close to it.

Gated RNNs: Long Short-Term Memory Networks

Long short-term memory (LSTM) networks alleviate the vanishing gradients problem by using *gate neurons* to control access to the hidden states.

Consider the m -dimensional hidden state vector $\mathbf{h}_t \in \mathbb{R}^m$ at time t . In a regular RNN, we update the hidden state as follows:

$$\mathbf{h}_t = f^h(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}_h)$$

Let $\mathbf{g} \in \{0,1\}^m$ be a binary vector. If we take the element-wise product of $\mathbf{g}$ and $\mathbf{h}_t$, namely, $\mathbf{g} \odot \mathbf{h}_t$, then elements of $\mathbf{g}$ act as gates that either allow the corresponding element of $\mathbf{h}_t$ to be retained or set to zero, that is, select elements of $\mathbf{h}_t$ to be remembered or forgotten.

However, for backpropagation we need *differentiable gates*, for which we use sigmoid activation on the gate neurons so that their value lies in the range $[0,1]$.

Like a logical gate, such neurons allow the inputs to be completely remembered if the value is 1, or forgotten if the value is 0.

In addition, they allow a weighted memory, allowing partial remembrance of the elements of $\mathbf{h}_t$, for values between 0 and 1.

Gated RNNs: Differentiable Gates

Consider a hidden state vector

$$\mathbf{h}_t = (-0.94 \quad 1.05 \quad 0.39 \quad 0.97 \quad 0.90)^T$$

and a logical gate vector $\mathbf{g}$:

$$\mathbf{g} = (0 \quad 1 \quad 1 \quad 0 \quad 1)^T$$

Their element-wise product gives

$$\mathbf{g} \odot \mathbf{h}_t = (0 \quad 1.05 \quad 0.39 \quad 0 \quad 0.90)^T$$

The first and fourth elements have been “forgotten.” Now consider a differentiable gate vector

$$\mathbf{g} = (0.1 \quad 0 \quad 1 \quad 0.9 \quad 0.5)^T$$

The element-wise product of $\mathbf{g}$ and $\mathbf{h}_t$ gives

$$\mathbf{g} \odot \mathbf{h}_t = (-0.094 \quad 0 \quad 0.39 \quad 0.873 \quad 0.45)^T$$

Only a fraction specified by an element of $\mathbf{g}$ is retained as a memory.

Gated RNNs: Long Short-Term Memory Networks

Forget Gate

To see how gated neurons work, we consider an RNN with a *forget gate*.

In a regular RNN, assuming tanh activation, the hidden state vector is updated unconditionally, as follows:

$$\mathbf{h}_t = \tanh(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b}_h)$$

Instead of directly updating $\mathbf{h}_t$, we will employ the forget gate neurons to control how much of the previous hidden state vector to forget when computing its new value, and also to control how to update it in light of the new input $\mathbf{x}_t$.

Given input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$, we first compute a candidate update vector $\mathbf{u}_t$, as follows:

$$\mathbf{u}_t = \tanh(\mathbf{W}_u^T \mathbf{x}_t + \mathbf{W}_{hu}^T \mathbf{h}_{t-1} + \mathbf{b}_u)$$

$\mathbf{u}_t$ is essentially the unmodified hidden state vector, as in a regular RNN.

Gated RNNs: Long Short-Term Memory Networks

Forget Gate

Using the forget gate, we can compute the new hidden state vector as follows:

$$\mathbf{h}_t = \phi_t \odot \mathbf{h}_{t-1} + (1 - \phi_t) \odot \mathbf{u}_t$$

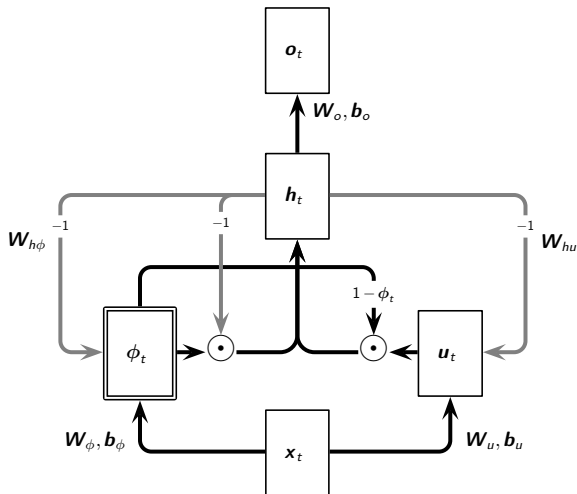
We can see that the new hidden state vector retains a fraction of the previous hidden state values, and a (complementary) fraction of the candidate update values.

Observe that if $\phi_t = 0$, i.e., if we want to entirely forget the previous hidden state, then $1 - \phi_t = 1$, which means that the hidden state will be updated completely at each time step just like in a regular RNN.

RNN with a forget gate ϕ_t

Recurrent connections shown in gray, forget gate shown doublelined.

$\odot$ denotes element wise product.



Gated RNNs: Long Short-Term Memory Networks

ϕ_t is a layer that $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$; these connections are fully connected, and are specified by the corresponding $\mathbf{W}_{h\phi}$ and $\mathbf{W}_{\phi}$, and $\mathbf{b}_{\phi}$.

ϕ_t needs to modify the previous hidden state layer $\mathbf{h}_{t-1}$, and therefore, both ϕ_t and $\mathbf{h}_{t-1}$ feed into what is essentially a new *element-wise* product layer, denoted by $\odot$.

Finally, the output of this element-wise product layer is used as input to the new hidden layer $\mathbf{h}_t$ that also takes input from another element-wise gate that computes the output from $\mathbf{u}_t$ and $1 - \phi_t$.

Thus, unlike regular layers that are fully connected and have a weight matrix and bias vector between the layers, the connections between ϕ_t and $\mathbf{h}_t$ via the element-wise layer are all one-to-one, and the weights are fixed at the value 1 with bias 0. Likewise the connections between $\mathbf{u}_t$ and $\mathbf{h}_t$ via the other element-wise layer are also one-to-one, with weights fixed at 1 and bias at 0.

Gated RNNs

Example

Let $m = 5$. Assume that the previous hidden state vector and the candidate update vector are given as follows:

$$\mathbf{h}_{t-1} = (-0.94 \quad 1.05 \quad 0.39 \quad 0.97 \quad 0.9)^T \quad \mathbf{u}_t = (0.5 \quad 2.5 \quad -1.0 \quad -0.5 \quad 0.8)^T$$

Let the forget gate and its complement be given as follows:

$$\phi_t = (0.9 \quad 1 \quad 0 \quad 0.1 \quad 0.5)^T \quad 1 - \phi_t = (0.1 \quad 0 \quad 1 \quad 0.9 \quad 0.5)^T$$

The new hidden state vector is then computed as the weighted sum of the previous hidden state vector and the candidate update vector:

$$\begin{aligned} \mathbf{h}_t &= \phi_t \odot \mathbf{h}_{t-1} + (1 - \phi_t) \odot \mathbf{u}_t \\ &= (0.9 \quad 1 \quad 0 \quad 0.1 \quad 0.5)^T \odot (-0.94 \quad 1.05 \quad 0.39 \quad 0.97 \quad 0.9)^T + \\ &\quad (0.1 \quad 0 \quad 1 \quad 0.9 \quad 0.5)^T \odot (0.5 \quad 2.5 \quad -1.0 \quad -0.5 \quad 0.8)^T \\ &= (-0.846 \quad 1.05 \quad 0 \quad 0.097 \quad 0.45)^T + (0.05 \quad 0 \quad -1.0 \quad -0.45 \quad 0.40)^T \\ &= (-0.796 \quad 1.05 \quad -1.0 \quad -0.353 \quad 0.85)^T \end{aligned}$$

Gated RNNs: Long Short-Term Memory Networks

Computing Net Gradients

The net gradients at the outputs are computed by considering the partial derivatives of the activation function ($\partial \mathbf{f}_t^o$) and the error function ($\partial \mathcal{E}_{\mathbf{x}_t}$):

$$\delta_t^o = \partial \mathbf{f}_t^o \odot \partial \mathcal{E}_{\mathbf{x}_t}$$

For the other layers, we can reverse all the arrows to determine the dependencies between the layers. The net gradient δ_{ti}^u at update layer neuron i at time t is:

$$\delta_{ti}^u = \frac{\partial \mathcal{E}_{\mathbf{x}}}{\partial \text{net}_{ti}^u} = \frac{\partial \mathcal{E}_{\mathbf{x}}}{\partial \text{net}_{ti}^h} \cdot \frac{\partial \text{net}_{ti}^h}{\partial u_{ti}} \cdot \frac{\partial u_{ti}}{\partial \text{net}_{ti}^u} = \delta_{ti}^h \cdot (1 - \phi_{ti}) \cdot (1 - u_{ti}^2)$$

where $\frac{\partial \text{net}_{ti}^h}{\partial u_{ti}} = \frac{\partial}{\partial u_{ti}} \{ \phi_{ti} \cdot h_{t-1,i} + (1 - \phi_{ti}) \cdot u_{ti} \} = 1 - \phi_{ti}$, and the update layer uses a tanh activation function.

Across all neurons, we obtain the net gradient at $\mathbf{u}_t$ as follows:

$$\delta_t^u = \delta_t^h \odot (1 - \phi_t) \odot (1 - \mathbf{u}_t \odot \mathbf{u}_t)$$

Gated RNNs: Long Short-Term Memory Networks

The net gradient δ_{ti}^ϕ at forget gate neuron i at time t is given as

$$\delta_{ti}^\phi = \frac{\partial \mathcal{E}_x}{\partial \text{net}_{ti}^\phi} = \frac{\partial \mathcal{E}_x}{\partial \text{net}_{ti}^h} \cdot \frac{\partial \text{net}_{ti}^h}{\partial \phi_{ti}} \cdot \frac{\partial \phi_{ti}}{\partial \text{net}_{ti}^\phi} = \delta_{ti}^h \cdot (h_{t-1,i} - u_{ti}) \cdot \phi_{ti}(1 - \phi_{ti})$$

Across all neurons, we obtain the net gradient at ϕ_t as follows:

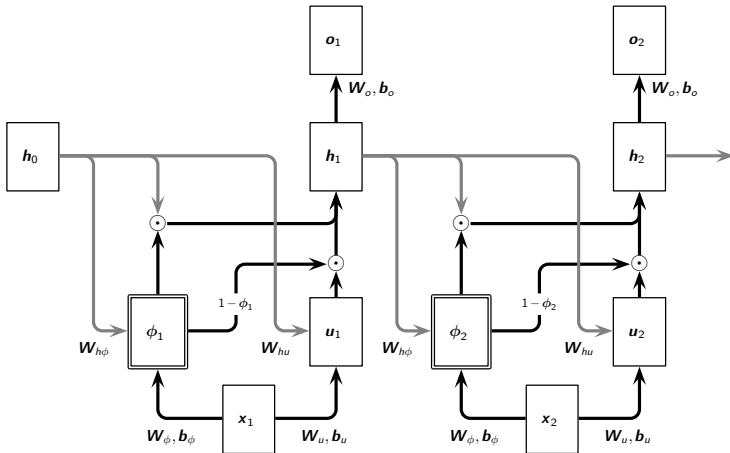
$$\delta_t^\phi = \delta_t^h \odot (\mathbf{h}_{t-1} - \mathbf{u}_t) \odot \phi_t \odot (1 - \phi_t)$$

Considering all the layers, including the output, forget, update and element-wise layers, the complete net gradient vector at the hidden layer at time t is given as:

$$\delta_t^h = \mathbf{W}_o \delta_t^o + \mathbf{W}_{h\phi} \delta_{t+1}^\phi + \mathbf{W}_{hu} \delta_{t+1}^u + (\delta_{t+1}^h \odot \phi_{t+1})$$

RNN with a forget gate unfolded in time

Recurrent connections in gray



Long Short-Term Memory (LSTM) Networks

LSTMs use differentiable gate vectors to control the hidden state vector $\mathbf{h}_t$, as well as another vector $\mathbf{c}_t \in \mathbb{R}^m$ called the *internal memory* vector.

In particular, LSTMs utilize three *gate vectors*: an input gate vector $\kappa_t \in \mathbb{R}^m$, a forget gate vector $\phi_t \in \mathbb{R}^m$, and an output gate vector $\omega_t \in \mathbb{R}^m$.

Like a regular RNN, an LSTM also maintains a hidden state vector for each time step. However, the content of the hidden vector is selectively copied from the internal memory vector via the output gate, with the internal memory being updated via the input gate and parts of it forgotten via the forget gate.

Each of the gate vectors conceptually plays a different role in an LSTM network. The input gate vector κ_t controls how much of the input vector, via the candidate update vector $\mathbf{u}_t$, is allowed to influence the memory vector $\mathbf{c}_t$.

The forget gate vector ϕ_t controls how much of the previous memory vector to forget, and finally the output gate vector ω_t controls how much of the memory state is retained for the hidden state.

LSTM neural network

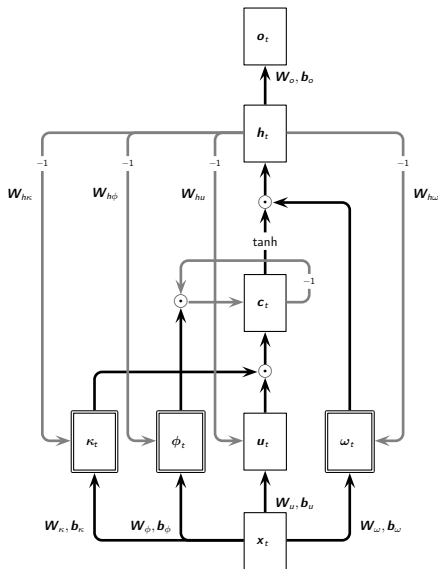
Recurrent connections shown in gray, gate layers shown doubledlined.

At each time step t , the three gate vectors are updated as follows:

$$\kappa_t = \sigma(W_{\kappa}^T x_t + W_{h\kappa}^T h_{t-1} + b_{\kappa})$$

$$\phi_t = \sigma(W_{\phi}^T x_t + W_{h\phi}^T h_{t-1} + b_{\phi})$$

$$\omega_t = \sigma(W_{\omega}^T x_t + W_{h\omega}^T h_{t-1} + b_{\omega})$$



LSTM neural network

Given $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$, an LSTM computes $\mathbf{u}_t$ after applying the tanh activation:

$$\mathbf{u}_t = \tanh(\mathbf{W}_u^T \mathbf{x}_t + \mathbf{W}_{hu}^T \mathbf{h}_{t-1} + \mathbf{b}_u)$$

It then computes the internal memory and hidden state vectors:

$$\begin{aligned}\mathbf{c}_t &= \kappa_t \odot \mathbf{u}_t + \phi_t \odot \mathbf{c}_{t-1} \\ \mathbf{h}_t &= \omega_t \odot \tanh(\mathbf{c}_t)\end{aligned}$$

$\mathbf{c}_t$ depends on $\mathbf{u}_t$ and $\mathbf{c}_{t-1}$. However, κ_t controls the extent to which $\mathbf{u}_t$ influences $\mathbf{c}_t$, and the forget gate ϕ_t controls how much of the previous memory is forgotten.

$\mathbf{h}_t$ depends on a tanh activated internal memory vector $\mathbf{c}_t$, but ω_t controls how much of the internal memory is reflected in the hidden state.

Finally, the output of the network $\mathbf{o}_t$ is obtained by applying the output activation function f^o to an affine combination of the hidden state neuron values:

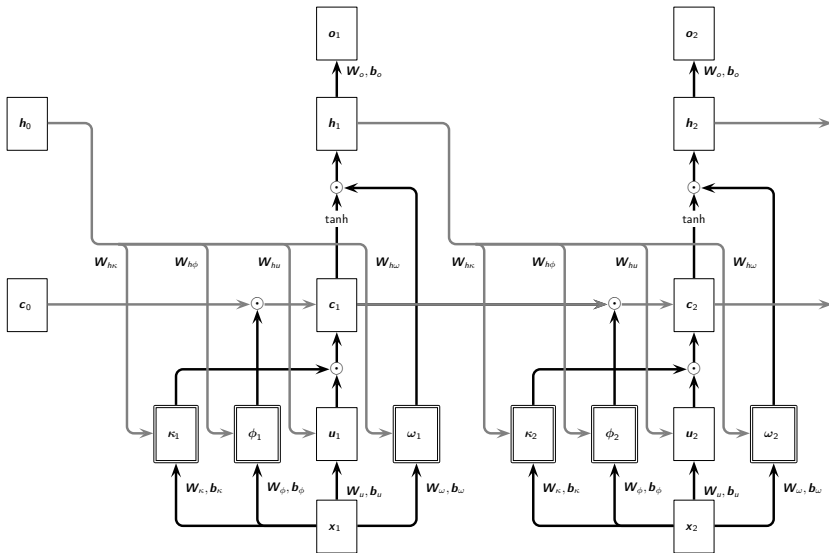
$$\mathbf{o}_t = f^o(\mathbf{W}_o^T \mathbf{h}_t + \mathbf{b}_o)$$

LSTMs can typically handle long sequences since the net gradients for the internal memory states do not vanish over long time steps. This is because, by design, the memory state $\mathbf{c}_{t-1}$ at time $t-1$ is linked to the memory state $\mathbf{c}_t$ at time t via implicit weights fixed at 1 and biases fixed at 0, with linear activation.

This allows the error to flow across time steps without vanishing or exploding.

LSTM neural network unfolded in time

Recurrent connections in gray



Training LSTMs

During backpropagation the net gradient vector at the output layer at time t is computed by considering the partial derivatives of the activation function, $\partial \mathbf{f}_t^o$, and the error function, $\partial \mathcal{E}_{x_t}$, as follows:

$$\delta_t^o = \partial \mathbf{f}_t^o \odot \partial \mathcal{E}_{x_t}$$

where we assume that the output neurons are independent. The net gradient vector δ_t^c at $\mathbf{c}_t$ is therefore given as:

$$\delta_t^c = \delta_t^h \odot \omega_t \odot (1 - \mathbf{c}_t \odot \mathbf{c}_t) + \delta_{t+1}^c \odot \phi_{t+1}$$

Across all forget gate neurons, the net gradient vector is therefore given as

$$\delta_t^\phi = \delta_t^c \odot \mathbf{c}_{t-1} \odot (1 - \phi_t) \odot \phi_t$$

The input gate also has only one incoming edge in backpropagation, from $\mathbf{c}_t$, via the element-wise multiplication $\kappa_t \odot \mathbf{u}_t$, with sigmoid activation. In a similar manner, as outlined above for δ_t^ϕ , the net gradient δ_t^κ at the input gate κ_t is:

$$\delta_t^\kappa = \delta_t^c \odot \mathbf{u}_t \odot (1 - \kappa_t) \odot \kappa_t$$

Training LSTMs

The same reasoning applies to the update candidate $\mathbf{u}_t$, which also has an incoming edge from $\mathbf{c}_t$ via $\kappa_t \odot \mathbf{u}_t$ and $\tanh$ activation, so the net gradient vector δ_t^u at the update layer is

$$\delta_t^u = \delta_t^c \odot \kappa_t \odot (1 - \mathbf{u}_t \odot \mathbf{u}_t)$$

Likewise, in backpropagation, there is one incoming connection to the output gate from $\mathbf{h}_t$ via $\omega_t \odot \tanh(\mathbf{c}_t)$ with sigmoid activation, therefore

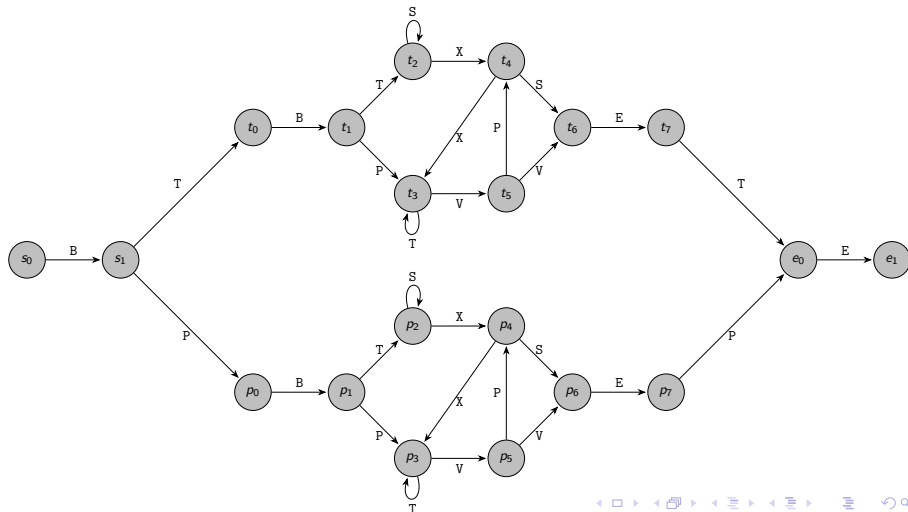
$$\delta_t^\omega = \delta_t^h \odot \tanh(\mathbf{c}_t) \odot (1 - \omega_t) \odot \omega_t$$

Finally, to compute the net gradients at the hidden layer, note that gradients flow back to $\mathbf{h}_t$ from the following layers: $\mathbf{u}_{t+1}, \kappa_{t+1}, \phi_{t+1}, \omega_{t+1}$ and $\mathbf{o}_t$. Therefore, the net gradient vector at the hidden state vector δ_t^h is given as

$$\delta_t^h = \mathbf{W}_o \delta_t^o + \mathbf{W}_{h\kappa} \delta_{t+1}^\kappa + \mathbf{W}_{h\phi} \delta_{t+1}^\phi + \mathbf{W}_{h\omega} \delta_{t+1}^\omega + \mathbf{W}_{hu} \delta_{t+1}^u$$

Example – Embedded Reber grammar automata

We use an LSTM to learn the embedded Reber grammar, which is generated according to the automata. This automata has two copies of the Reber automata



LSTM

Example – Embedded Reber grammar automata

From the state s_1 , the top automata is reached by following the edge labeled T, whereas the bottom automata is reached via the edge labeled P. The states of the top automata are labeled as $t_0, t_1, \dots, t_7$, whereas the states of the bottom automata are labeled as $p_0, p_1, \dots, p_7$. Finally, note that the state e_0 can be reached from either the top or the bottom automata by following the edges labeled T and P, respectively.

The first symbol is always B and the last symbol is always E. However, the important point is that the second symbol is always the same as the second last symbol, and thus any sequence learning model has to learn this long range dependency. For example, the following is a valid embedded Reber sequence: $S_{\mathcal{X}} = \langle B, T, B, T, S, S, X, X, T, V, V, E, T, E \rangle$.

The task of the LSTM is to learn to predict the next symbol for each of the positions in a given embedded Reber sequence.

LSTM

Example – Embedded Reber grammar automata

For training, we generate $n = 400$ embedded Reber sequences with a minimum length of 40, and convert them into training pairs $(\mathcal{X}, \mathcal{Y})$ using the binary encoding. The maximum sequence length is $\tau = 64$.

Given the long range dependency, we used an LSTM with $m = 20$ hidden neurons (smaller values of m either need more epochs to learn, or have trouble learning the grammar). The input and output layer sizes are determined by the dimensionality of encoding, namely $d = 7$ and $p = 7$. We use sigmoid activation at the output layer, treating each neuron as independent.

Finally, we use the binary cross entropy error function. The LSTM is trained for $r = 10000$ epochs (using step size $\eta = 1$ and batch size 400); it learns the training data perfectly, making no errors in the prediction of the set of possible next symbols.

LSTM

Example – Embedded Reber grammar automata

We test the LSTM model on 100 previously unseen embedded Reber sequences (with minimum length 40, as before). The trained LSTM makes no errors on the test sequences. In particular, it is able to learn the long range dependency between the second symbol and the second last symbol, which must always match.

The embedded Reber grammar was chosen since an RNN has trouble learning the long range dependency. Using an RNN with $m = 60$ hidden neurons, using $r = 25000$ epochs with a step size of $\eta = 1$, the RNN can perfectly learn the training sequences. That is, it makes no errors on any of the 400 training sequences.

However, on the test data, this RNN makes a mistake in 40 out of the 100 test sequences. In fact, in each of these test sequences it makes exactly one error; it fails to correctly predict the second last symbol.

These results suggest that while the RNN is able to “memorize” the long range dependency in the training data, it is not able to generalize completely on unseen test sequences.

Convolutional Neural Networks

A convolutional neural network (CNN) is essentially a *localized* and sparse feedforward MLP that is designed to exploit spatial and/or temporal structure in the input data.

In a regular MLP all of the neurons in layer l are connected to all of the neurons in layer $l + 1$.

In contrast, a CNN connects a contiguous or adjacent subset of neurons in layer l to a single neuron in the next layer $l + 1$.

Different sliding windows comprising contiguous subsets of neurons in layer l connect to different neurons in layer $l + 1$.

Furthermore, all of these sliding windows use *parameter sharing*, that is, the same set of weights, called a *filter*, is used for all sliding windows. Finally, different filters are used to automatically extract features from layer l for use by layer $l + 1$.

1D Convolution

Given a vector $\mathbf{a} \in \mathbb{R}^k$, define the summation operator as one that adds all the elements of the vector. That is,

$$\text{sum}(\mathbf{a}) = \sum_{i=1}^k a_i$$

A 1D convolution between $\mathbf{x}$ and $\mathbf{w}$, denoted by the asterisk symbol $*$, is:

$$\mathbf{x} * \mathbf{w} = \left(\text{sum}(\mathbf{x}_k(1) \odot \mathbf{w}) \quad \cdots \quad \text{sum}(\mathbf{x}_k(n-k+1) \odot \mathbf{w}) \right)^T$$

where $\odot$ is the element-wise product, so that

$$\text{sum}(\mathbf{x}_k(i) \odot \mathbf{w}) = \sum_{j=1}^k x_{i+j-1} \cdot w_j$$

for $i = 1, 2, \dots, n - k + 1$. The convolution of $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{w} \in \mathbb{R}^k$ results in a vector of length $n - k + 1$.

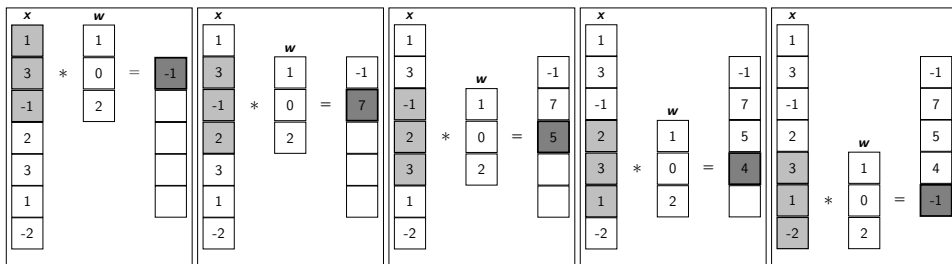
1D Convolution

Given $\mathbf{x}$ with $n=7$ and a filter $\mathbf{w} = (1, 0, 2)^T$ ($k=3$). The first window of $\mathbf{x}$ is $\mathbf{x}_3(1) = (1, 3, -1)^T$.

$$\text{sum}(\mathbf{x}_3(1) \odot \mathbf{w}) = \text{sum}((1, 3, -1)^T \odot (1, 0, 2)^T) = \text{sum}((1, 0, -2)^T) = -1$$

The convolution $\mathbf{x} * \mathbf{w}$ has size $n - k + 1 = 7 - 3 + 1 = 5$, and is given as

$$\mathbf{x} * \mathbf{w} = (-1, 7, 5, 4, -1)^T$$



2D Convolution

Given a $k \times k$ matrix $\mathbf{A} \in \mathbb{R}^{k \times k}$, define the summation operator as one that adds all the elements of the matrix. That is,

$$\text{sum}(\mathbf{A}) = \sum_{i=1}^k \sum_{j=1}^k a_{i,j}$$

The *2D convolution* of $\mathbf{X}$ and $\mathbf{W}$, denoted $\mathbf{X} * \mathbf{W}$, is defined as:

$$\mathbf{X} * \mathbf{W} = \begin{pmatrix} \text{sum}(\mathbf{X}_k(1,1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(1,n-k+1) \odot \mathbf{W}) \\ \vdots & \cdots & \vdots \\ \text{sum}(\mathbf{X}_k(n-k+1,1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(n-k+1,n-k+1) \odot \mathbf{W}) \end{pmatrix}$$

where $\odot$ is the element-wise product of $\mathbf{X}_k(i,j)$ and $\mathbf{W}$, so that

$$\text{sum}(\mathbf{X}_k(i,j) \odot \mathbf{W}) = \sum_{a=1}^k \sum_{b=1}^k x_{i+a-1,j+b-1} \cdot w_{a,b}$$

for $i, j = 1, 2, \dots, n-k+1$. The convolution of $\mathbf{X} \in \mathbb{R}^{n \times n}$ and $\mathbf{W} \in \mathbb{R}^{k \times k}$ results in a $(n-k+1) \times (n-k+1)$ matrix.

2D Convolution

Example

Given matrix $\mathbf{X}$ with $n = 4$ and a filter $\mathbf{W}$ with window size $k = 2$. The convolution of the first window of $\mathbf{X}$, namely $\mathbf{X}_2(1,1)$, with $\mathbf{W}$ is given as:

$$\text{sum}(\mathbf{X}_2(1,1) \odot \mathbf{W}) = \text{sum} \left(\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} \odot \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) = \text{sum} \left(\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) = 2$$

The convolution steps for different 2×2 sliding windows of $\mathbf{X}$ with the filter $\mathbf{W}$ are shown.

The convolution $\mathbf{X} * \mathbf{W}$ has size 3×3 , since $n - k + 1 = 4 - 2 + 1 = 3$, and is given as

$$\mathbf{X} * \mathbf{W} = \begin{pmatrix} 2 & 6 & 4 \\ 4 & 4 & 8 \\ 4 & 4 & 4 \end{pmatrix}$$

2D Convolution

X

1	2	2	1
3	1	4	2
2	1	3	4
1	2	3	1

W

1	0
0	1

$*$

2		

X

1	2	2	1
3	1	4	2
2	1	3	4
1	2	3	1

W

1	0
0	1

$*$

2	6	

X

1	2	2	1
3	1	4	2
2	1	3	4
1	2	3	1

W

1	0
0	1

$*$

2	6	4

Diagram illustrating the calculation of the dot product for the first row of matrix X and the first column of matrix W .

Matrix X (4x4):

1	2	2	1
3	1	4	2
2	1	3	4
1	2	3	1

Matrix W (2x2):

1	0
0	1

Result (4x2):

2	6	4
4		

$$\begin{array}{|c|c|c|c|} \hline & \mathbf{X} & & \\ \hline 1 & 2 & 2 & 1 \\ \hline 3 & 1 & 4 & 2 \\ \hline 2 & 1 & 3 & 4 \\ \hline 1 & 2 & 3 & 1 \\ \hline \end{array}
 \quad * \quad
 \begin{array}{|c|c|} \hline & \mathbf{W} \\ \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|} \hline 2 & 6 & 4 \\ \hline 4 & 4 & \\ \hline & & \\ \hline \end{array}$$

X

1	2	2	1
3	1	4	2
2	1	3	4
1	2	3	1

W

1	0
0	1

$\ast$

2	6	4
4	4	8

$$\begin{array}{|c|c|c|c|} \hline \mathbf{X} \\ \hline 1 & 2 & 2 & 1 \\ \hline 3 & 1 & 4 & 2 \\ \hline 2 & 1 & 3 & 4 \\ \hline 1 & 2 & 3 & 1 \\ \hline \end{array} * \begin{array}{|c|c|} \hline \mathbf{W} \\ \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 2 & 6 & 4 \\ \hline 4 & 4 & 8 \\ \hline 4 & & \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|} \hline & \mathbf{X} & & \\ \hline 1 & 2 & 2 & 1 \\ \hline 3 & 1 & 4 & 2 \\ \hline 2 & 1 & 3 & 4 \\ \hline 1 & 2 & 3 & 1 \\ \hline \end{array}
 \quad * \quad
 \begin{array}{|c|c|} \hline & \mathbf{W} \\ \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|} \hline 2 & 6 & 4 \\ \hline 4 & 4 & 8 \\ \hline 4 & 4 & \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|} \hline \mathbf{X} \\ \hline 1 & 2 & 2 & 1 \\ \hline 3 & 1 & 4 & 2 \\ \hline 2 & 1 & 3 & 4 \\ \hline 1 & 2 & 3 & 1 \\ \hline \end{array}
 \quad * \quad
 \begin{array}{|c|c|} \hline \mathbf{W} \\ \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array}
 =
 \begin{array}{|c|c|c|} \hline 2 & 6 & 4 \\ \hline 4 & 4 & 8 \\ \hline 4 & 4 & 4 \\ \hline \end{array}$$

3D Convolution

Let $\mathbf{W}$ be a $k \times k \times r$ tensor of weights, called a *3D filter*, with $k \leq n$ and $r \leq m$. Let $\mathbf{X}_k(i, j, q)$ denote the $k \times k \times r$ subtensor of $\mathbf{X}$ starting at row i , column j and channel q , as illustrated in the figure, with $1 \leq i, j \leq n - k + 1$, and $1 \leq q \leq m - r + 1$.

Given a $k \times k \times r$ tensor $\mathbf{A} \in \mathbb{R}^{k \times k \times r}$, define the summation operator as one that adds all the elements of the tensor. That is,

$$\text{sum}(\mathbf{A}) = \sum_{i=1}^k \sum_{j=1}^k \sum_{q=1}^r a_{i,j,q}$$

where $a_{i,j,q}$ is the element of $\mathbf{A}$ at row i , column j , and channel q .

$$\text{sum}(\mathbf{X}_k(i, j, q) \odot \mathbf{W}) = \sum_{a=1}^k \sum_{b=1}^k \sum_{c=1}^r x_{i+a-1, j+b-1, q+c-1} \cdot w_{a,b,c}$$

for $i, j = 1, 2, \dots, n - k + 1$ and $q = 1, 2, \dots, m - r + 1$.

The convolution of $\mathbf{X}$ and $\mathbf{W}$ results in a $(n - k + 1) \times (n - k + 1) \times (m - r + 1)$ tensor.

3D Convolution

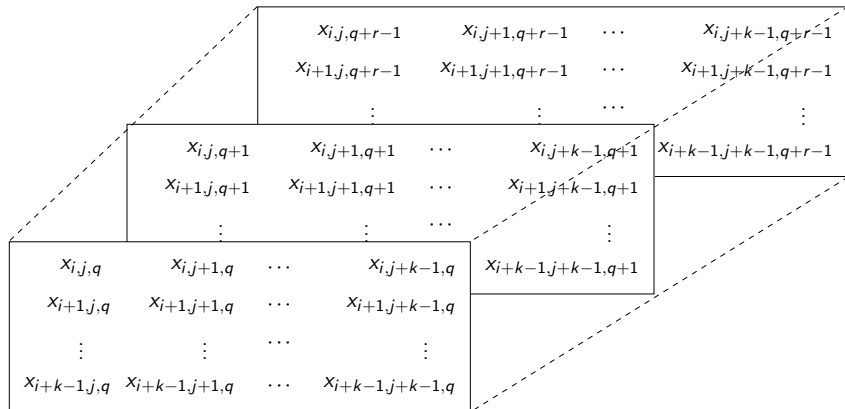
The *3D convolution* of $\mathbf{X}$ and $\mathbf{W}$, denoted $\mathbf{X} * \mathbf{W}$, is defined as:

$$\mathbf{X} * \mathbf{W} = \begin{pmatrix} \begin{pmatrix} \text{sum}(\mathbf{X}_k(1,1,1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(1,n-k+1,1) \odot \mathbf{W}) \\ \text{sum}(\mathbf{X}_k(2,1,1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(2,n-k+1,1) \odot \mathbf{W}) \\ \vdots & \cdots & \vdots \\ \text{sum}(\mathbf{X}_k(n-k+1,1,1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(n-k+1,n-k+1,1) \odot \mathbf{W}) \end{pmatrix} \\ \hline \begin{pmatrix} \text{sum}(\mathbf{X}_k(1,1,2) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(1,n-k+1,2) \odot \mathbf{W}) \\ \text{sum}(\mathbf{X}_k(2,1,2) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(2,n-k+1,2) \odot \mathbf{W}) \\ \vdots & \cdots & \vdots \\ \text{sum}(\mathbf{X}_k(n-k+1,1,2) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(n-k+1,n-k+1,2) \odot \mathbf{W}) \end{pmatrix} \\ \hline \begin{pmatrix} \text{sum}(\mathbf{X}_k(1,1,m-r+1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(1,n-k+1,m-r+1) \odot \mathbf{W}) \\ \text{sum}(\mathbf{X}_k(2,1,m-r+1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(2,n-k+1,m-r+1) \odot \mathbf{W}) \\ \vdots & \cdots & \vdots \\ \text{sum}(\mathbf{X}_k(n-k+1,1,m-r+1) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{X}_k(n-k+1,n-k+1,m-r+1) \odot \mathbf{W}) \end{pmatrix} \end{pmatrix}$$

where $\odot$ is the element-wise product of $\mathbf{X}_k(i,j,q)$ and $\mathbf{W}$.

3D subtensor $\mathbf{X}_k(i, j, q)$

$k \times k \times r$ subtensor of $\mathbf{X}$ starting at row i , column j , and channel q .



3D Convolution

Example

Given a $3 \times 3 \times 3$ tensor $\mathbf{X}$ with $n = 3$ and $m = 3$, and a $2 \times 2 \times 3$ filter $\mathbf{W}$ with window size $k = 2$ and $r = 3$.

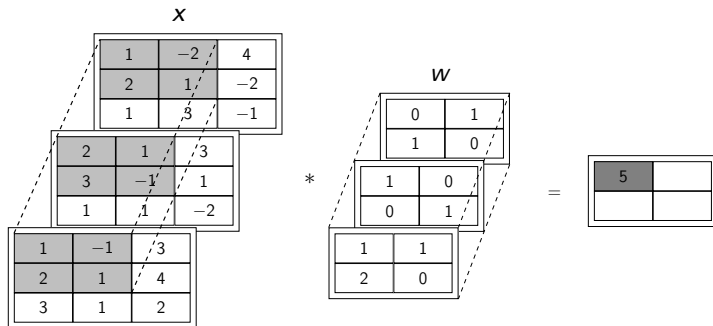
The convolution of the first window of $\mathbf{X}$, namely $\mathbf{X}_2(1,1)$, with $\mathbf{W}$ is given as

$$\begin{aligned}\text{sum}(\mathbf{X}_2(1,1) \odot \mathbf{W}) &= \text{sum} \left(\left(\begin{array}{cc|cc} 1 & -1 & 2 & 1 \\ 2 & 1 & 3 & -1 \end{array} \right) \odot \left(\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 2 & 0 & 0 & 1 \end{array} \right) \right) \\ &= \text{sum} \left(\left(\begin{array}{cc|cc} 1 & -1 & 2 & 0 \\ 4 & 0 & 0 & -1 \end{array} \right) \right) = 5\end{aligned}$$

where we stack the different channels horizontally.

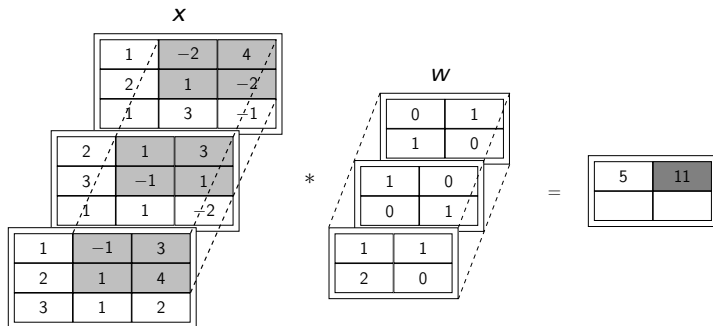
3D Convolution

Given a $3 \times 3 \times 3$ tensor $\mathbf{X}$ with $n=3$ and $m=3$, and a $2 \times 2 \times 3$ filter $\mathbf{W}$ with $k=2$ and $r=3$, the upper-left element is:



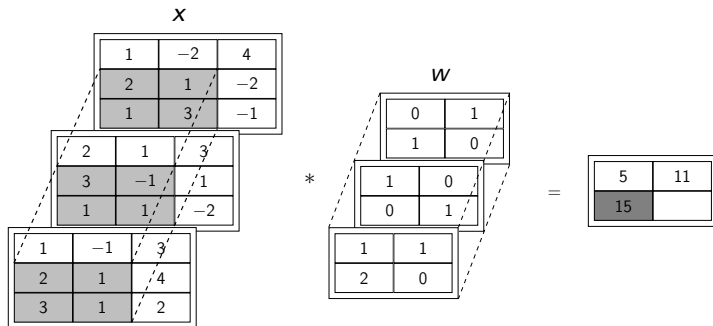
3D Convolution

Given a $3 \times 3 \times 3$ tensor $\mathbf{X}$ with $n=3$ and $m=3$, and a $2 \times 2 \times 3$ filter $\mathbf{W}$ with $k=2$ and $r=3$, the upper-right element is:

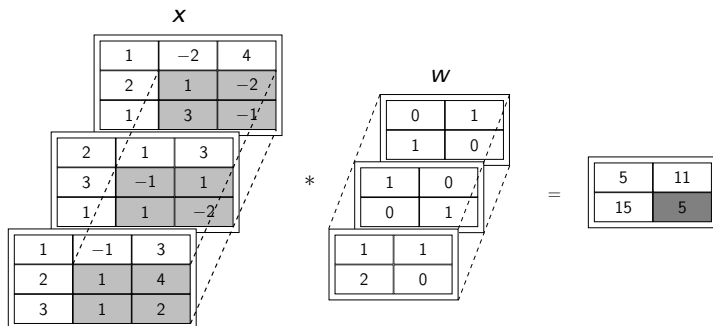


3D Convolution

Given a $3 \times 3 \times 3$ tensor $\mathbf{X}$ with $n=3$ and $m=3$, and a $2 \times 2 \times 3$ filter $\mathbf{W}$ with $k=2$ and $r=3$, the bottom-left element is:



3D Convolution



The convolution $X * W$ has size 2×2 , since $n - k + 1 = 3 - 2 + 1 = 2$ and $r = m = 3$; it is given as

$$X * W = \begin{pmatrix} 5 & 11 \\ 15 & 5 \end{pmatrix}$$

Bias and Activation Functions

Filter Bias

Let $\mathbf{W}$ be a $k \times k \times m_l$ 3D filter. Recall that when we convolve $\mathbf{Z}^l$ and $\mathbf{W}$, we get a $(n_l - k + 1) \times (n_l - k + 1)$ matrix at layer $l + 1$.

However, so far, we have ignored the role of the bias term in the convolution. Let $b \in \mathbb{R}$ be a scalar bias value for $\mathbf{W}$, and let $\mathbf{Z}_k^l(i, j)$ denote the $k \times k \times m_l$ subtensor of $\mathbf{Z}^l$ at position (i, j) .

$$net_{i,j}^{l+1} = \text{sum}(\mathbf{Z}_k^l(i, j) \odot \mathbf{W}) + b$$

and the value of the neuron $z_{i,j}^{l+1}$ is obtained by applying some f

$$z_{i,j}^{l+1} = f\left(\text{sum}(\mathbf{Z}_k^l(i, j) \odot \mathbf{W}) + b\right)$$

The activation function can be any of the ones typically used in neural networks, for example, identity, sigmoid, tanh, ReLU and so on.

Bias and Activation Functions

Multiple 3D Filters

We can observe that one 3D filter $\mathbf{W}$ with a corresponding bias term b results in a $(n_l - k + 1) \times (n_l - k + 1)$ matrix of in layer $l + 1$.

If we desire m_{l+1} channels in layer $l + 1$, then we need m_{l+1} different $k \times k \times m_l$ filters $\mathbf{W}_q$ with a corresponding b_q , to obtain the $(n_l - k + 1) \times (n_l - k + 1) \times m_{l+1}$ tensor at layer $l + 1$:

$$\mathbf{Z}^{l+1} = \left\{ z_{i,j,q}^{l+1} = f \left(\text{sum}(\mathbf{Z}_k^l(i,j) \odot \mathbf{W}_q) + b_q \right) \right\}_{i,j=1,2,\dots,n_l-k+1 \text{ and } q=1,2,\dots,m_{l+1}}$$

In summary, a convolution layer takes as input the $n_l \times n_l \times m_l$ tensor $\mathbf{Z}^l$ of neurons from layer l , and then computes the $n_{l+1} \times n_{l+1} \times m_{l+1}$ tensor $\mathbf{Z}^{l+1}$ of neurons for the next layer $l + 1$ via the convolution of $\mathbf{Z}^l$ with a set of m_{l+1} different 3D filters of size $k \times k \times m_l$, followed by adding the bias and applying some non-linear activation function f .

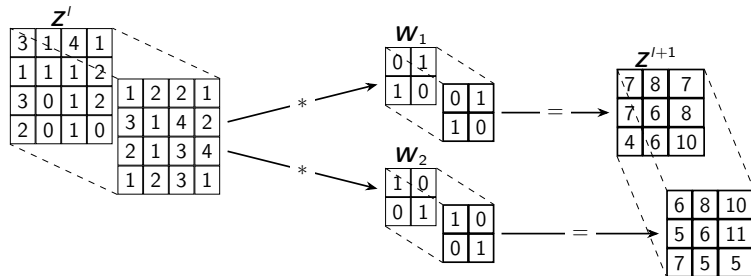
Note that each 3D filter applied to $\mathbf{Z}^l$ results in a new channel in layer $l + 1$. Therefore, m_{l+1} filters are used to yield m_{l+1} channels at layer $l + 1$.

Multiple 3D filters.

Example shows a $4 \times 4 \times 2$ tensor $\mathbf{Z}^I$ with $n = 4$ and $m = 2$. It also shows two different $2 \times 2 \times 2$ filters $\mathbf{W}_1$ and $\mathbf{W}_2$ with $k = 2$ and $r = 2$.

Since $r = m = 2$, the convolution of $\mathbf{Z}^I$ and $\mathbf{W}_i$ (for $i = 1, 2$) results in a 3×3 matrix since $n - k + 1 = 4 - 2 + 1 = 3$.

However, $\mathbf{W}_1$ yields one channel and $\mathbf{W}_2$ yields a second channel, so that the tensor for the next layer $\mathbf{Z}^{I+1}$ has size $3 \times 3 \times 2$, with two channels (one per filter).



One of the issues with the convolution operation is that the size of the tensors will necessarily decrease in each successive CNN layer.

If layer l has size $n_l \times n_l \times m_l$, and we use filters of size $k \times k \times m_l$, then each channel in layer $l + 1$ will have size $(n_l - k + 1) \times (n_l - k + 1)$.

That is the number of rows and columns for each successive tensor will shrink by $k - 1$ and that will limit the number of layers the CNN can have.

Padding

To get around this limitation, a simple solution is to pad each tensor along both the rows and columns in each channel by some default value, typically zero.

For uniformity, we always pad by adding the same number of rows at the top and at the bottom, and likewise the same number of columns on the left and on the right.

With padding p , the size of layer l tensor is then $(n_l + 2p) \times (n_l + 2p) \times m_l$. Assume that each filter is of size $k \times k \times m_l$, and assume there are m_{l+1} filters, then the size of the layer $l + 1$ tensor will be $(n_l + 2p - k + 1) \times (n_l + 2p - k + 1) \times m_{l+1}$. Since we want to preserve the size of the resulting tensor:

$$n_l + 2p - k + 1 \geq n_l, \text{ which implies, } p = \left\lceil \frac{k - 1}{2} \right\rceil$$

With padding, we can have arbitrarily deep convolutional layers in a CNN.

Padding 2D Convolution

$$p=0 \text{ e } p=1$$

Example shows a 2D convolution without and with padding. It starts with a convolution of a 5×5 matrix Z^l ($n=5$) with a 3×3 filter W ($k=3$), which results in a 3×3 matrix since $n - k + 1 = 5 - 3 + 1 = 3$. Thus, the size of the next layer Z^{l+1} has decreased.

This way, we can chain together as many convolution layers as desired, without decrease in the size of the layers.

$$\begin{array}{|c|c|c|c|c|} \hline & & Z^l & & \\ \hline 1 & 2 & 2 & 1 & 1 \\ \hline 3 & 1 & 4 & 2 & 1 \\ \hline 2 & 1 & 3 & 4 & 3 \\ \hline 1 & 2 & 3 & 1 & 1 \\ \hline 4 & 1 & 3 & 2 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ \hline 0 & 1 & 1 \\ \hline 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline & Z^{l+1} & \\ \hline 7 & 11 & 9 \\ \hline 9 & 11 & 12 \\ \hline 8 & 8 & 7 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|c|c|c|} \hline & & Z^l & & & & \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & 2 & 2 & 1 & 1 & 0 \\ \hline 0 & 3 & 1 & 4 & 2 & 1 & 0 \\ \hline 0 & 2 & 1 & 3 & 4 & 3 & 0 \\ \hline 0 & 1 & 2 & 3 & 1 & 1 & 0 \\ \hline 0 & 4 & 1 & 3 & 2 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ \hline 0 & 1 & 1 \\ \hline 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|c|c|} \hline & & Z^{l+1} & & \\ \hline 6 & 5 & 7 & 4 & 2 \\ \hline 6 & 7 & 11 & 9 & 5 \\ \hline 4 & 9 & 11 & 12 & 6 \\ \hline 7 & 8 & 8 & 7 & 6 \\ \hline 5 & 5 & 7 & 6 & 2 \\ \hline \end{array}$$

Striding is often used to sparsify the number of sliding windows used in the convolutions.

Instead of considering all possible windows we increment the index along both rows and columns by an integer value $s \geq 1$ called the *stride*.

A 3D convolution of $\mathbf{Z}^l$ ($n_l \times n_l \times m_l$) with a $\mathbf{W}$ ($k \times k \times m_l$), using stride s :

$$\mathbf{Z}^l * \mathbf{W} = \begin{pmatrix} \text{sum}(\mathbf{Z}_k^l(1,1) \odot \mathbf{W}) & \text{sum}(\mathbf{Z}_k^l(1,1+s) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{Z}_k^l(1,1+t \cdot s) \odot \mathbf{W}) \\ \text{sum}(\mathbf{Z}_k^l(1+s,1) \odot \mathbf{W}) & \text{sum}(\mathbf{Z}_k^l(1+s,1+s) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{Z}_k^l(1+s,1+t \cdot s) \odot \mathbf{W}) \\ \vdots & \vdots & \cdots & \vdots \\ \text{sum}(\mathbf{Z}_k^l(1+t \cdot s,1) \odot \mathbf{W}) & \text{sum}(\mathbf{Z}_k^l(1+t \cdot s,1+s) \odot \mathbf{W}) & \cdots & \text{sum}(\mathbf{Z}_k^l(1+t \cdot s,1+t \cdot s) \odot \mathbf{W}) \end{pmatrix}$$

where $t = \left\lfloor \frac{n_l - k}{s} \right\rfloor$.

We can observe that using stride s , the convolution of $\mathbf{Z}^l \in \mathbb{R}^{n_l \times n_l \times m_l}$ with $\mathbf{W} \in \mathbb{R}^{k \times k \times m_l}$ results in a $(t+1) \times (t+1)$ matrix.

Striding

Example

Example shows 2D convolution using stride $s = 2$ on a 5×5 matrix $\mathbf{Z}^l$ ($n_l = 5$) with a filter $\mathbf{W}$ of size 3×3 ($k = 3$).

Instead of the default stride of one, which would result in a 3×3 matrix, we get a $(t+1) \times (t+1) = 2 \times 2$ matrix $\mathbf{Z}^{l+1}$, since

$$t = \left\lfloor \frac{n_l - k}{s} \right\rfloor = \left\lfloor \frac{5 - 3}{2} \right\rfloor = 1$$

We can see that the next window index increases by s along the rows and columns.

For example, the first window is $\mathbf{Z}_3^l(1,1)$ and thus the second window is $\mathbf{Z}_3^l(1,1+s) = \mathbf{Z}_3^l(1,3)$. Next, we move down by a stride of $s = 2$, so that the third window is $\mathbf{Z}_3^l(1+s,1) = \mathbf{Z}_3^l(3,1)$, and the final window is $\mathbf{Z}_3^l(3,1+s) = \mathbf{Z}_3^l(3,3)$

Striding

Example: $s = 2$

$$\begin{array}{|c|c|c|c|c|} \hline & Z^I & & & \\ \hline 1 & 2 & 2 & 1 & 1 \\ 3 & 1 & 4 & 2 & 1 \\ 2 & 1 & 3 & 4 & 3 \\ 1 & 2 & 3 & 1 & 1 \\ 4 & 1 & 3 & 2 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline & Z^{I+1} & \\ \hline 7 & \\ \hline & \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|c|} \hline & Z^I & & & \\ \hline 1 & 2 & 2 & 1 & 1 \\ 3 & 1 & 4 & 2 & 1 \\ 2 & 1 & 3 & 4 & 3 \\ 1 & 2 & 3 & 1 & 1 \\ 4 & 1 & 3 & 2 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline & Z^{I+1} & \\ \hline 7 & 9 \\ \hline & \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|c|} \hline & Z^I & & & \\ \hline 1 & 2 & 2 & 1 & 1 \\ 3 & 1 & 4 & 2 & 1 \\ 2 & 1 & 3 & 4 & 3 \\ 1 & 2 & 3 & 1 & 1 \\ 4 & 1 & 3 & 2 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline & Z^{I+1} & \\ \hline 7 & 9 \\ 8 & \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|c|c|} \hline & Z^I & & & \\ \hline 1 & 2 & 2 & 1 & 1 \\ 3 & 1 & 4 & 2 & 1 \\ 2 & 1 & 3 & 4 & 3 \\ 1 & 2 & 3 & 1 & 1 \\ 4 & 1 & 3 & 2 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline & W & \\ \hline 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline & Z^{I+1} & \\ \hline 7 & 9 \\ 8 & 7 \\ \hline \end{array}$$

CNNs also use other types of aggregation functions in addition to summation, such as average and maximum.

Avg-Pooling average over the element-wise product of $\mathbf{Z}_k^l(i, j, q)$ and $\mathbf{W}$:

$$\begin{aligned}\text{avg}(\mathbf{Z}_k^l(i, j, q) \odot \mathbf{W}) &= \text{avg}_{\substack{a=1,2,\dots,k \\ b=1,2,\dots,k \\ c=1,2,\dots,r}} \{z_{i+a-1,j+b-1,q+c-1}^l \cdot w_{a,b,c}\} \\ &= \frac{1}{k^2 \cdot r} \cdot \text{sum}(\mathbf{Z}_k^l(i, j, q) \odot \mathbf{W})\end{aligned}$$

Max-Pooling maximum over the element-wise product of $\mathbf{Z}_k^l(i, j, q)$ and $\mathbf{W}$:

$$\max(\mathbf{Z}_k^l(i, j, q) \odot \mathbf{W}) = \max_{\substack{a=1,2,\dots,k \\ b=1,2,\dots,k \\ c=1,2,\dots,r}} \{z_{i+a-1,j+b-1,q+c-1}^l \cdot w_{a,b,c}\}$$

Max-Pooling in CNNs

Typically, max-pooling is used more often than avg-pooling. Also, for pooling it is very common to set the stride equal to the filter size ($s = k$), so that the aggregation function is applied over disjoint $k \times k$ windows in each channel in $\mathbf{Z}^l$.

More importantly, in pooling, the filter $\mathbf{W}$ is by default taken to be a $k \times k \times 1$ tensor all of whose weights are fixed as 1, so that $\mathbf{W} = \mathbf{1}_{k \times k \times 1}$.

In other words, the filter weights are fixed at 1 and are not updated during backpropagation. Further, the filter uses a fixed zero bias (that is, $b = 0$).

Finally, note that pooling implicitly uses an identity activation function. As such, the convolution of $\mathbf{Z}^l \in \mathbb{R}^{n_l \times n_l \times m_l}$ with $\mathbf{W} \in \mathbb{R}^{k \times k \times 1}$, using stride $s = k$, results in a tensor $\mathbf{Z}^{l+1}$ of size $\lfloor \frac{n_l}{s} \rfloor \times \lfloor \frac{n_l}{s} \rfloor \times m_l$.

Max-Pooling

Example

Example shows max-pooling on a 4×4 matrix $\mathbf{Z}^l$ ($n_l = 4$), using window size $k = 2$ and stride $s = 2$ that equals the window size.

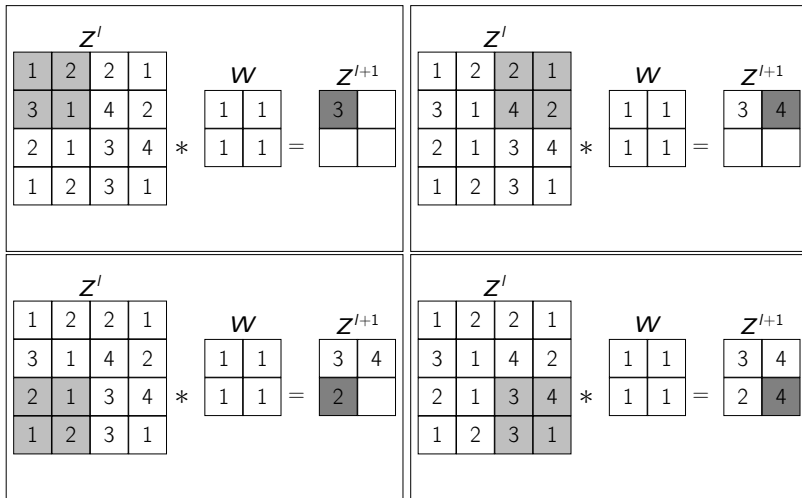
The resulting layer $\mathbf{Z}^{l+1}$ thus has size 2×2 , since $\lfloor \frac{n_l}{s} \rfloor = \lfloor \frac{4}{2} \rfloor = 2$. We can see that the filter $\mathbf{W}$ has fixed weights equal to 1.

The convolution of the first window of $\mathbf{Z}^l$, namely $\mathbf{Z}_2^l(1,1)$, with $\mathbf{W}$ is given as

$$\max(\mathbf{Z}_2^l(1,1) \odot \mathbf{W}) = \max\left(\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} \odot \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}\right) = \max\left(\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}\right) = 3$$

Max-pooling

Example – Stride: $s = 2$



In a typical CNN architecture, one alternates between a convolution layer (with summation as the aggregation function, and learnable filter weights and bias term) and a pooling layer (say, with max-pooling and fixed filter of ones).

The intuition is that, whereas the convolution layer learns the filters to extract informative features, the pooling layer applies an aggregation function like max (or avg) to extract the most important neuron value (or the mean of the neuron values) within each sliding window, in each of the channels.

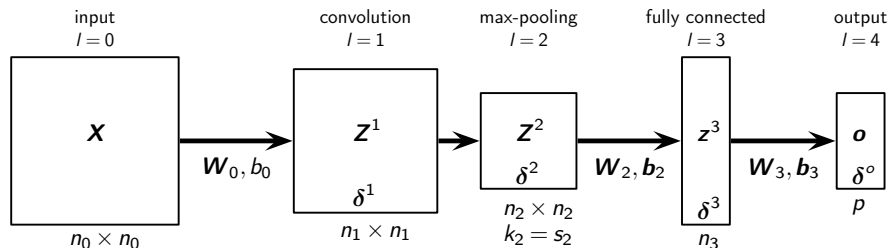
Starting from the input layer, a deep CNN is comprised of multiple, typically alternating, convolution and pooling layers, followed by one or more fully connected layers, and then the final output layer.

For each convolution and pooling layer we need to choose the window size k as well as the stride value s , and whether to use padding p or not. We also have to choose the non-linear activation functions for the convolution layers, and also the number of layers to consider.

Training: Convolutional neural network

To see how to train CNNs we will consider a network with a single convolution layer and a max-pooling layer, followed by a fully connected layer.

For simplicity, we assume that there is only one channel for the input $\mathbf{X}$, and further, we use only one filter.



Deep CNNs

Feed-forward Phase

Let $\mathbf{D} = \{\mathbf{X}_i, \mathbf{y}_i\}_{i=1}^n$ denote the training data, comprising n tensors $\mathbf{X}_i \in \mathbb{R}^{n_0 \times n_0 \times m_0}$ (with $m_0 = 1$ for ease of explanation) and the corresponding response vector $\mathbf{y}_i \in \mathbb{R}^p$.

Given a training pair $(\mathbf{X}, \mathbf{y}) \in \mathbf{D}$, in the feed-forward phase, the predicted output $\mathbf{o}$ is given via the following equations:

$$\mathbf{Z}^1 = f^1((\mathbf{X} * \mathbf{W}_0) + \mathbf{b}_0)$$

$$\mathbf{Z}^2 = \mathbf{Z}^1 *_{s_2, \max} \mathbf{1}_{k_2 \times k_2}$$

$$\mathbf{z}^3 = f^3(\mathbf{W}_2^T \mathbf{z}^2 + \mathbf{b}_2)$$

$$\mathbf{o} = f^o(\mathbf{W}_o^T \mathbf{z}^3 + \mathbf{b}_o)$$

where $*_{s_2, \max}$ denotes max-pooling with stride s_2 .

Let δ^1 , δ^2 , and δ^3 denote the net gradient vectors at layers $l = 1, 2, 3$, respectively, and let δ^o denote the net gradient vector at the output layer.

The output net gradient vector is obtained in the regular manner by computing the partial derivatives of the loss function ($\partial \mathcal{E}_x$) and the activation function (∂f^o):

$$\delta^o = \partial f^o \odot \partial \mathcal{E}_x$$

assuming that the output neurons are independent.

Since layer $l = 3$ is fully connected to the output layer, and likewise the max-pooling layer $l = 2$ is fully connected to $\mathbf{Z}^3$, the net gradients at these layers are computed as in a regular MLP

$$\delta^3 = \partial f^3 \odot (\mathbf{W}_o \cdot \delta^o)$$

$$\delta^2 = \partial f^2 \odot (\mathbf{W}_2 \cdot \delta^3) = \mathbf{W}_2 \cdot \delta^3$$

Let last step follows from the fact that $\partial \mathbf{f}^2 = 1$, since max-pooling implicitly uses an identity activation function. Consider the net gradient δ_{ij}^1 at neuron z_{ij}^1 in layer $l = 1$ where $i, j = 1, 2, \dots, n_1$.

$$\delta_{ij}^1 = \begin{cases} \delta_{ab}^2 \cdot \partial f_{ij}^1 & \text{if } i = i^* \text{ and } j = j^* \\ 0 & \text{otherwise} \end{cases}$$

In other words, the net gradient at neuron z_{ij}^1 in the convolution layer is zero if this neuron does not have the maximum value in its window.

Otherwise, if it is the maximum, the net gradient backpropagates from the max-pooling layer to this neuron and is then multiplied by the partial derivative of the activation function.

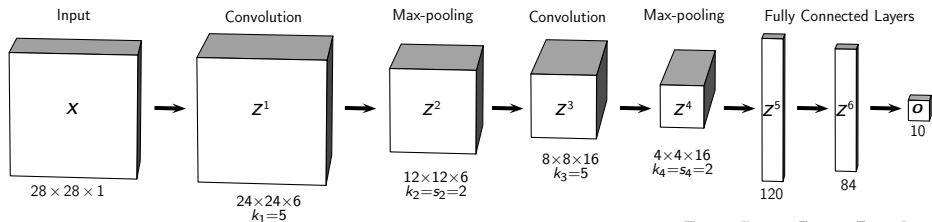
The $n_1 \times n_1$ matrix of net gradients δ^1 comprises the net gradients δ_{ij}^1 for all $i, j = 1, 2, \dots, n_1$.

Convolutional Neural Network

Example

Example shows a CNN for handwritten digit recognition. This CNN is trained and tested on the MNIST dataset, that contains 60,000 training images and 10,000 test images.

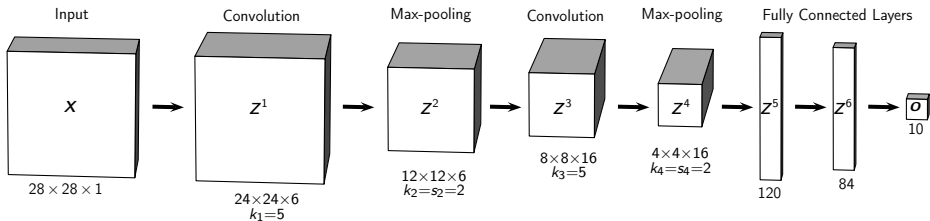
Each input image is a 28×28 matrix of pixel values between 0 to 255, which are divided by 255, so that each pixel lies in the interval $[0,1]$. The corresponding (true) output $\mathbf{y}_i$ is a one-hot encoded binary vector that denotes a digit from 0 to 9; the digit 0 is encoded as $\mathbf{e}_1 = (1,0,0,0,0,0,0,0,0,0)^T$, the digit 1 as $\mathbf{e}_2 = (0,1,0,0,0,0,0,0,0,0)^T$, and so on.



Convolutional Neural Network

Example

In our CNN model, all the convolution layers use stride equal to one, and do not use any padding, whereas all of the max-pooling layers use stride equal to the window size. Since each input is a 28×28 pixels image of a digit with 1 channel (grayscale), we have $n_0 = 28$ and $m_0 = 1$, and therefore, the input $\mathbf{X} = \mathbf{Z}^0$ is a $n_0 \times n_0 \times m_0 = 28 \times 28 \times 1$ tensor. The first convolution layer uses $m_1 = 6$ filters, with $k_1 = 5$ and stride $s_1 = 1$, without padding.

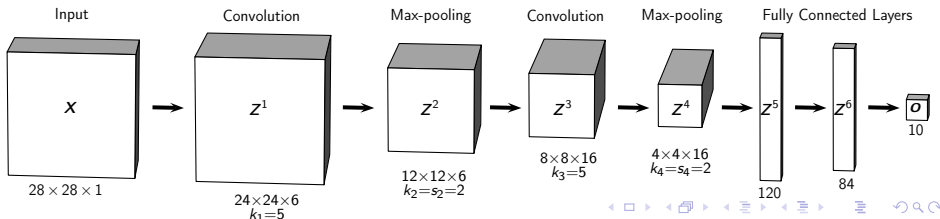


Convolutional Neural Network

Example

Each filter is a $5 \times 5 \times 1$ tensor of weights, and across the six filters the resulting layer $l = 1$ tensor $\mathbf{Z}^1$ has size $24 \times 24 \times 6$, with $n_1 = n_0 - k_1 + 1 = 28 - 5 + 1 = 24$ and $m_1 = 6$. The second hidden layer is a max-pooling ($k_2 = 2$ and $s_2 = 2$).

Since max-pooling by default uses a fixed filter $\mathbf{W} = 1_{k_2 \times k_2 \times 1}$, the resulting tensor $\mathbf{Z}^2$ has size $12 \times 12 \times 6$, with $n_2 = \left\lfloor \frac{n_1}{k_2} \right\rfloor = \left\lfloor \frac{24}{2} \right\rfloor = 12$, and $m_2 = 6$. The third layer is a convolution layer with $m_3 = 16$ channels, with a window size of $k_3 = 5$ (and stride $s_3 = 1$), resulting in the tensor $\mathbf{Z}^3$ of size $8 \times 8 \times 16$, where $n_3 = n_2 - k_3 + 1 = 12 - 5 + 1 = 8$.



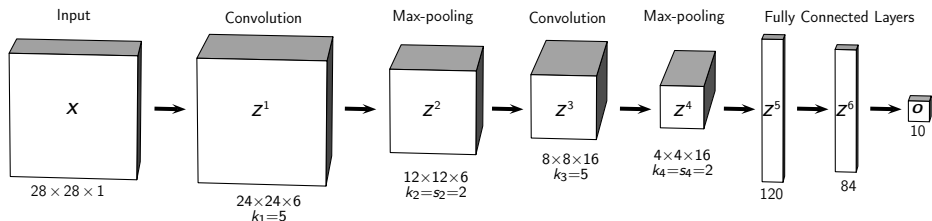
Convolutional Neural Network

Example

This is followed by another max-pooling layer that uses $k_4 = 2$ and $s_4 = 2$, which yields the tensor $\mathbf{Z}^4$ that is $4 \times 4 \times 16$, where $n_4 = \left\lfloor \frac{n_3}{k_4} \right\rfloor = \left\lfloor \frac{8}{2} \right\rfloor = 4$, and $m_4 = 16$.

The next three layers are fully connected as in a regular MLP. All of the $4 \times 4 \times 16 = 256$ neurons in layer $l = 4$ are connected to $l = 5$. $\mathbf{Z}^5$ is simply a vector and it can be considered a degenerate tensor of size $120 \times 1 \times 1$.

Layer $l = 5$ is also fully connected to layer $l = 6$ with 84 neurons. $\mathbf{o}$ has 10 neurons and uses softmax. $\mathbf{Z}^1$, $\mathbf{Z}^3$, $\mathbf{Z}^5$, and $\mathbf{Z}^6$ use ReLU activation.



Convolutional Neural Network

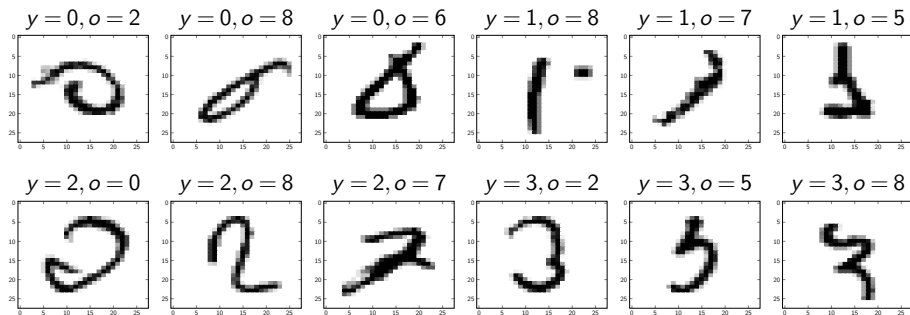
Example

We train the CNN model on $n = 60000$ training images from the MNIST dataset; we train for 15 epochs using step size $\eta = 0.2$ and using cross-entropy error (since there are 10 classes).

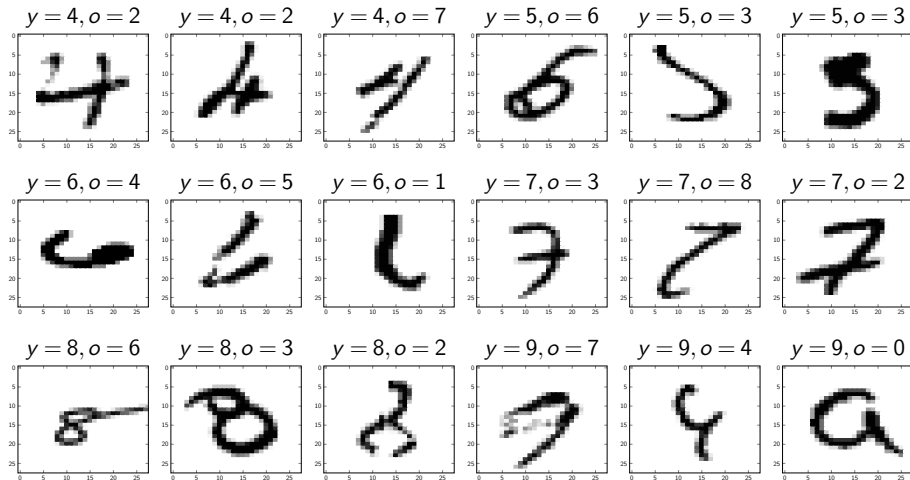
Training was done using minibatches, using batch size of 1000. After training the CNN model, we evaluate it on the test dataset of 10,000 images. The CNN model makes 147 errors on the test set, resulting in an error rate of 1.47%.

MNIST: Incorrect predictions

Figure shows examples of images that are misclassified by the CNN. We can see that several of the misclassified images are noisy, incomplete or erroneous, and hard to classify correctly even by a human.



MNIST: Incorrect predictions



Convolutional Neural Network

Example

For comparison, we also train a deep MLP with two (fully connected) hidden layers with the same sizes as the two fully connected layers before the output layer in the CNN shown in figure.

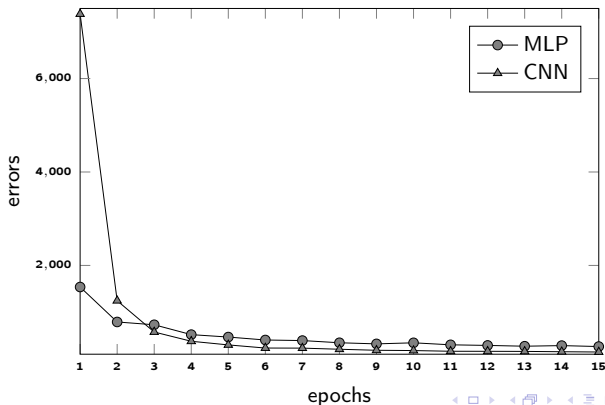
Therefore, the MLP comprises the layers $\mathbf{X}$, $\mathbf{Z}^5$, $\mathbf{Z}^6$, and $\mathbf{o}$, with the input 28×28 images viewed as a vector of size $d = 784$. The first hidden layer has size $n_1 = 120$, the second hidden layer has size $n_2 = 84$, and the output layer has size $p = 10$. We use ReLU activation function for all layers, except the output, which uses softmax.

We train the MLP model for 15 epochs on the training dataset with $n = 60000$ images, using step size $\eta = 0.5$. On the test dataset, the MLP made 264 errors, for an error rate of 2.64%.

MNIST: CNN versus Deep MLP

Prediction error as a function of epochs.

Graph shows the number of errors on the test set after each epoch of training for both the CNN and MLP model; the CNN model achieves significantly better accuracy than the MLP.



Regularization

Regularization is an approach whereby we constrain the model parameters to reduce overfitting, by reducing the variance at the cost of increasing the bias slightly.

In general, for any learning model M , if $L(\mathbf{y}, \hat{\mathbf{y}})$ is some loss function for a given input $\mathbf{x}$, and Θ denotes all the model parameters, where $\hat{\mathbf{y}} = M(\mathbf{x}|\Theta)$, the objective is to find the parameters that minimize the loss over all instances:

$$\min_{\Theta} J(\Theta) = \sum_{i=1}^n L(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \sum_{i=1}^n L(\mathbf{y}_i, M(\mathbf{x}_i|\Theta))$$

With regularization, we add a penalty on the parameters Θ :

$$\min_{\Theta} J(\Theta) = \sum_{i=1}^n L(\mathbf{y}_i, \hat{\mathbf{y}}_i) + \alpha R(\Theta)$$

where $\alpha \geq 0$ is the regularization constant.

L2 Regularization for Deep Learning

We first consider the case of a multilayer perceptron with one hidden layer, and then generalize it. The parameters of the model are

$$\Theta = \{\mathbf{W}_h, \mathbf{b}_h, \mathbf{W}_o, \mathbf{b}_o\}$$

The L_2 regularized objective is therefore given as

$$\min_{\Theta} J(\Theta) = \mathcal{E}_x + \frac{\alpha}{2} \cdot R_{L_2}(\mathbf{W}_o, \mathbf{W}_h) = \mathcal{E}_x + \frac{\alpha}{2} \cdot \left(\|\mathbf{W}_h\|_F^2 + \|\mathbf{W}_o\|_F^2 \right)$$

The regularized objective tries to minimize the individual weights for pairs of neurons. This adds some bias, but reduces variance, since small weights are more robust to changes in the input data in terms of the predicted output values. The gradient update rule is given as

$$\begin{aligned} \mathbf{W}_o &= \mathbf{W}_o - \eta \cdot \nabla_{\mathbf{W}_o} = \mathbf{W}_o - \eta \cdot (\mathbf{z} \cdot \delta_o^T + \alpha \cdot \mathbf{W}_o) = \mathbf{W}_o - \eta \cdot \alpha \cdot \mathbf{W}_o - \eta \cdot (\mathbf{z} \cdot \delta_o^T) \\ &= (1 - \eta \cdot \alpha) \cdot \mathbf{W}_o - \eta \cdot (\mathbf{z} \cdot \delta_o^T) \end{aligned}$$

L_2 regularization is also called *weight decay*, since the updated weight matrix uses decayed weights from the previous step, using the decay factor $1 - \eta \cdot \alpha$.

Given the error function $\mathcal{E}_x$, the L_2 regularized objective function is

$$\min_{\Theta} J(\Theta) = \mathcal{E}_x + \frac{\alpha}{2} \cdot R_{L_2}(\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_h) = \mathcal{E}_x + \frac{\alpha}{2} \cdot \left(\sum_{l=0}^h \|\mathbf{W}_l\|_F^2 \right)$$

where the parameters of the model are $\Theta = \{\mathbf{W}_0, \mathbf{b}_0, \mathbf{W}_1, \mathbf{b}_1, \dots, \mathbf{W}_h, \mathbf{b}_h\}$.

Based on the derivation for the one hidden layer, the regularized gradient is:

$$\nabla_{\mathbf{W}_l} = \mathbf{z}^l \cdot (\delta^{l+1})^T + \alpha \cdot \mathbf{W}_l$$

and the update rule for weight matrices is

$$\mathbf{W}_l = \mathbf{W}_l - \eta \cdot \nabla_{\mathbf{W}_l} = (1 - \eta \cdot \alpha) \cdot \mathbf{W}_l - \eta \cdot (\mathbf{z}^l \cdot (\delta^{l+1})^T)$$

for $l = 0, 1, \dots, h$, where δ^l is the net gradient vector for the hidden layer l .

Dropout Regularization

The idea behind dropout regularization is to randomly set a certain fraction of the neuron values in a layer to zero during training time. The aim is to make the network more robust and to avoid overfitting at the same time. By dropping random neurons for each training point, the network is forced to not rely on any specific set of edges.

From the perspective of a given neuron, since it cannot rely on all its incoming edges to be present, it has the effect of not concentrating the weight on specific input edges, but rather the weight is spread out among the incoming edges.

The net effect is similar to L_2 regularization since weight spreading leads to smaller weights on the edges. The resulting model with dropout is therefore more resilient to small perturbations in the input, which can reduce overfitting at a small price in increased bias.

However, note that while L_2 regularization directly changes the objective function, dropout regularization is a form of *structural regularization* that does not change the objective function, but instead changes the network topology in terms of which connections are currently active or inactive.

Dropout Regularization

MLP with One Hidden Layer

During the training phase, for each input $\mathbf{x}$, we create a random mask vector to drop a fraction of the hidden neurons. Formally, let $r \in [0, 1]$ be the probability of keeping a neuron, so that the dropout probability is $1 - r$.

We create a m -dimensional multivariate Bernoulli vector $\mathbf{u} \in \{0, 1\}^m$, called the *masking vector*, each of whose entries is 0 with dropout probability $1 - r$, and 1 with probability r . Let $\mathbf{u} = (u_1, u_2, \dots, u_m)^T$, where

$$u_i = \begin{cases} 0 & \text{with probability } 1 - r \\ 1 & \text{with probability } r \end{cases}$$

The feed-forward step is then given as

$$\mathbf{z} = f^h(\mathbf{b}_h + \mathbf{W}_h^T \mathbf{x})$$

$$\tilde{\mathbf{z}} = \mathbf{u} \odot \mathbf{z}$$

$$\mathbf{o} = f^o(\mathbf{b}_o + \mathbf{W}_o^T \tilde{\mathbf{z}})$$

where $\odot$ is the element-wise multiplication.

Inverted Dropout

There is one complication in the basic dropout approach above, namely, the expected output of hidden layer neurons is different during training and testing, since dropout is not applied during the testing phase (after all, we do not want the predictions to be randomly varying on a given test input). With r as the probability of retaining a hidden neuron, its expected output value is

$$E[z_i] = r \cdot z_i + (1 - r) \cdot 0 = r \cdot z_i$$

On the other hand, since there is no dropout at test time, the outputs of the hidden neurons will be higher at testing time. So one idea is to scale the hidden neuron values by r at testing time.

On the other hand, there is a simpler approach called *inverted dropout* that does not need a change at testing time. The idea is to rescale the hidden neurons after the dropout step during the training phase, as follows:

$$\mathbf{z} = f(\mathbf{b}_h + \mathbf{W}_h^T \mathbf{x}) \quad \tilde{\mathbf{z}} = \frac{1}{r} \cdot (\mathbf{u} \odot \mathbf{z}) \quad \mathbf{o} = f(\mathbf{b}_o + \mathbf{W}_o^T \tilde{\mathbf{z}})$$

Dropout in Deep MLPs

Dropout regularization for deep MLPs is done in a similar manner. Let $r_l \in [0, 1]$, for $l = 1, 2, \dots, h$ denote the probability of retaining a hidden neuron for layer l , so that $1 - r_l$ is the dropout probability. One can also use a single rate r for all the layers by setting $r_l = r$. Define the masking vector for hidden layer l , $\mathbf{u}^l \in \{0, 1\}^{n_l}$, as follows:

$$u_i^l = \begin{cases} 0 & \text{with probability } 1 - r_l \\ 1 & \text{with probability } r_l \end{cases}$$

The feed-forward step between layer l and $l + 1$ is then given as

$$\begin{aligned} \mathbf{z}^l &= f(\mathbf{b}_l + \mathbf{W}_l^T \tilde{\mathbf{z}}^{l-1}) \\ \tilde{\mathbf{z}}^l &= \frac{1}{r_l} \cdot (\mathbf{u}^l \odot \mathbf{z}^l) \end{aligned}$$

using inverted dropout.

Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 26: Deep Learning